

Intro to convolutional neural networks (CNNs) for medical image analysis

Matthew Engelhard

Today

- What can computer vision (CV) do, and how is it relevant to clinical medicine?
- The convolution operation
- Pooling and other architectural details
- CNNs for other data types (i.e., not images)
- Specific architectures (time permitting)

What can computer vision do?

Classification, Detection, Segmentation...

Binary Classification



**Diabetic
retinopathy?
(Yes/No)**

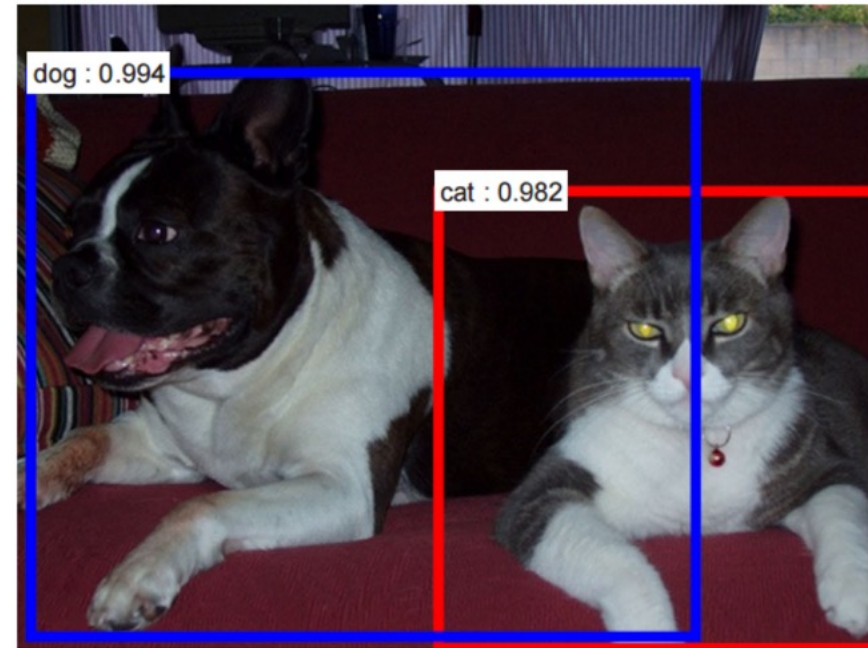
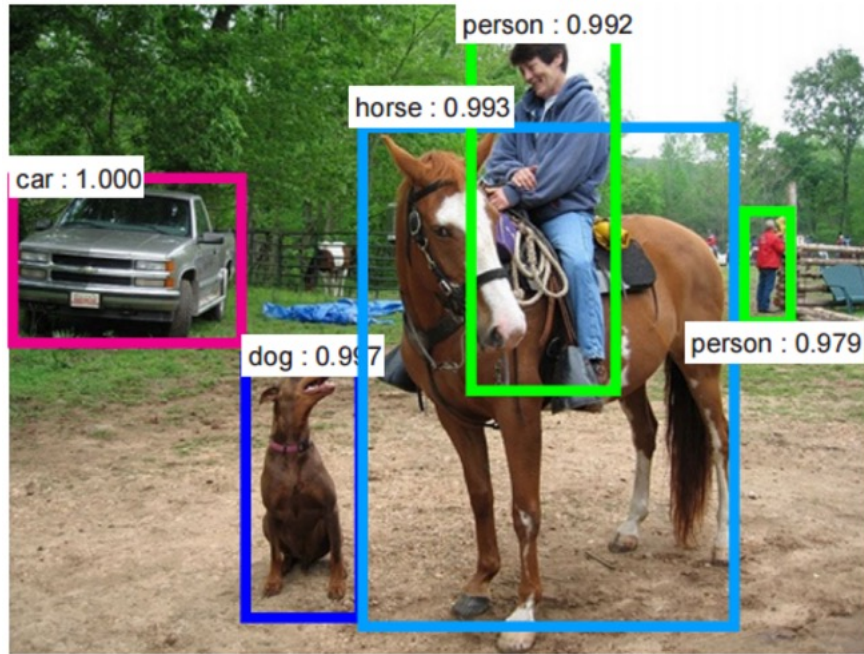
Multi-Class Classification



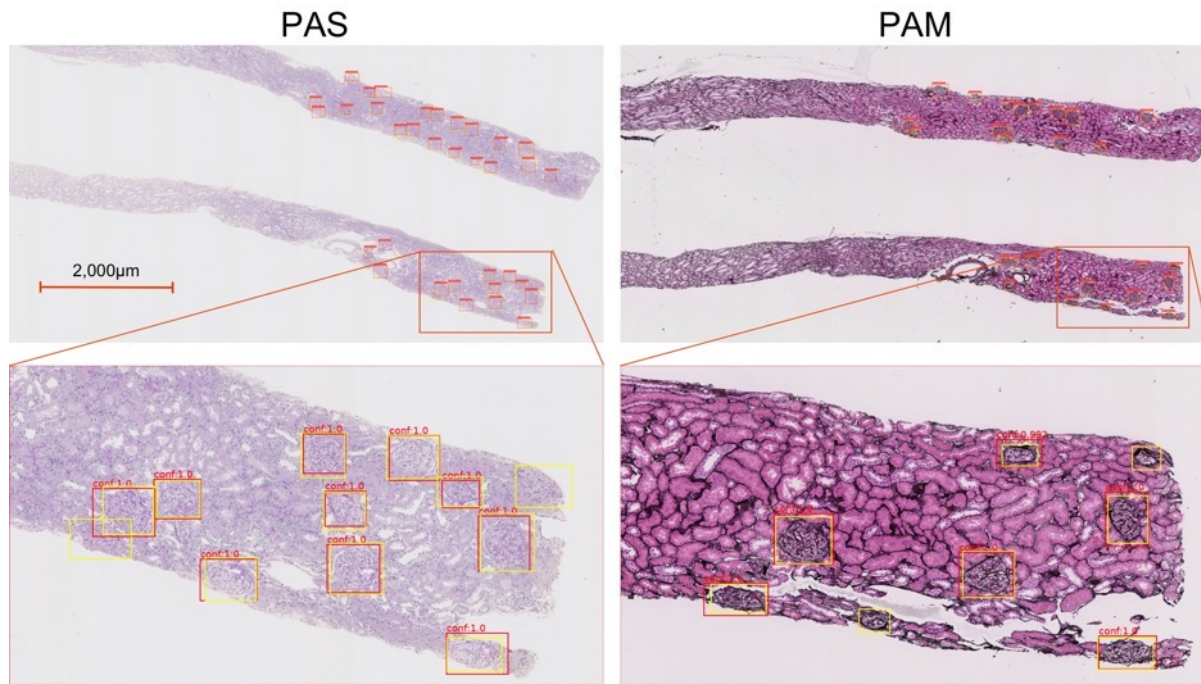
Dermatologist-level classification of skin cancer

Nature volume 542, pages 115–118 (02 February 2017)

Object Detection

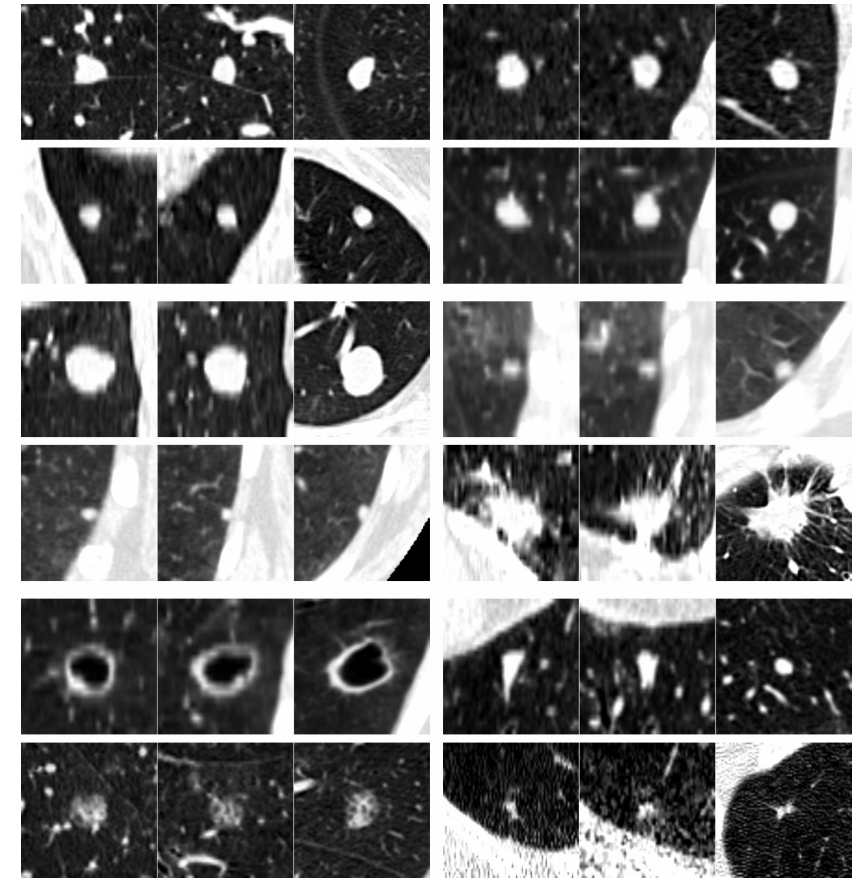


Object Detection in Medicine



Glomerular Detection with Faster-RCNN

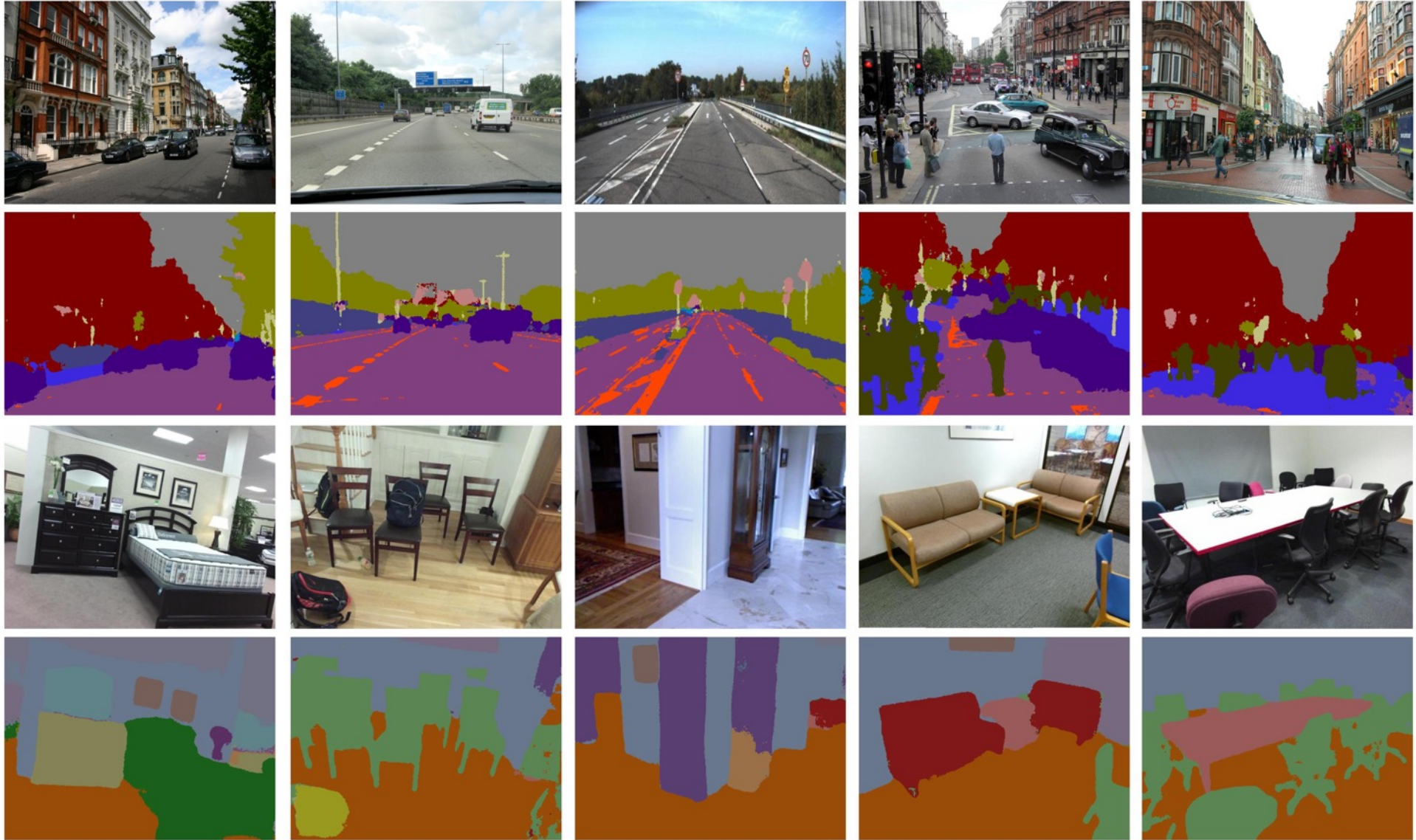
Kawazoe et al., *J. Imaging*, 2018



Pulmonary Nodule detection in CT

van Ginneken et al., *Biomedical Imaging*, 2015

Semantic Segmentation



Segment Anything (even unfamiliar objects)

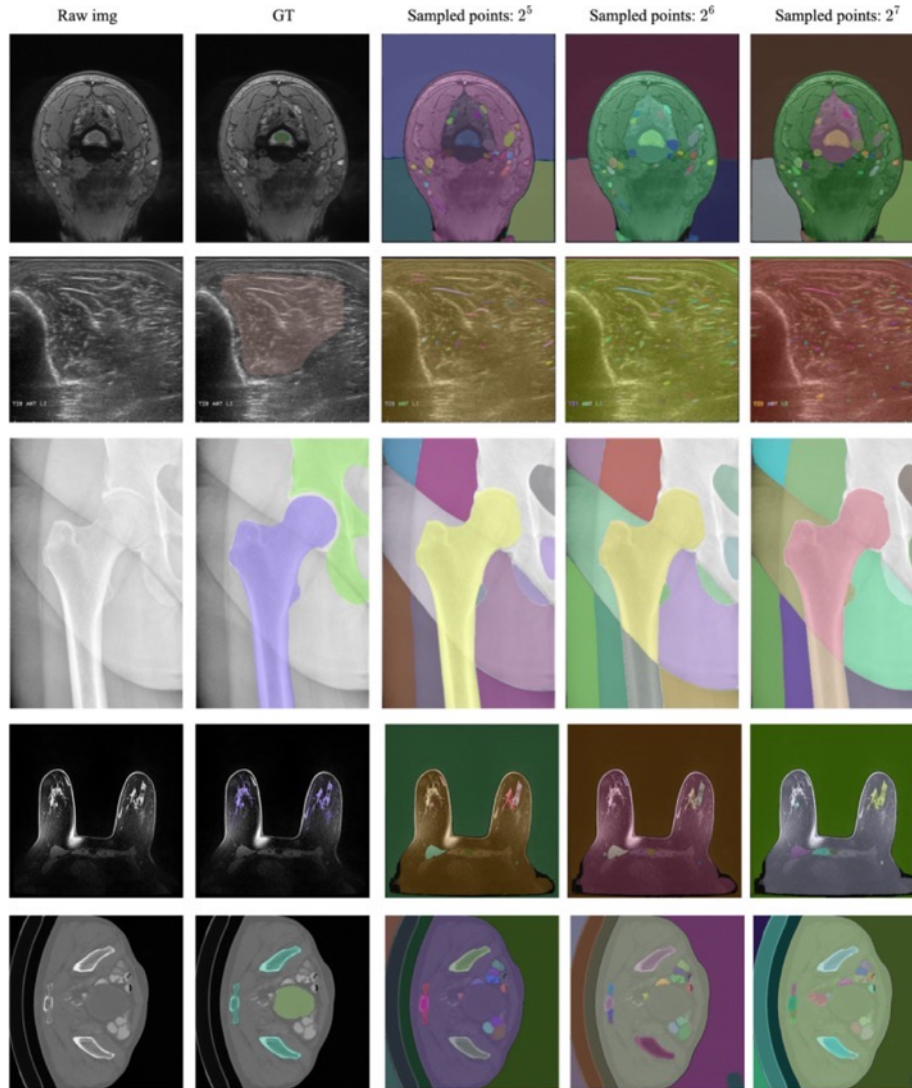


Kirillov, Alexander, et al. "Segment anything." *arXiv preprint arXiv:2304.02643* (2023).

<https://segment-anything.com>

-> Other Vision Applications

Segment Anything in Medical Images



Use Cases:

- Precise identification of boundaries
- Determine areas or volumes
- Calculate features useful for subsequent analyses/models
- Remove unwanted areas of images

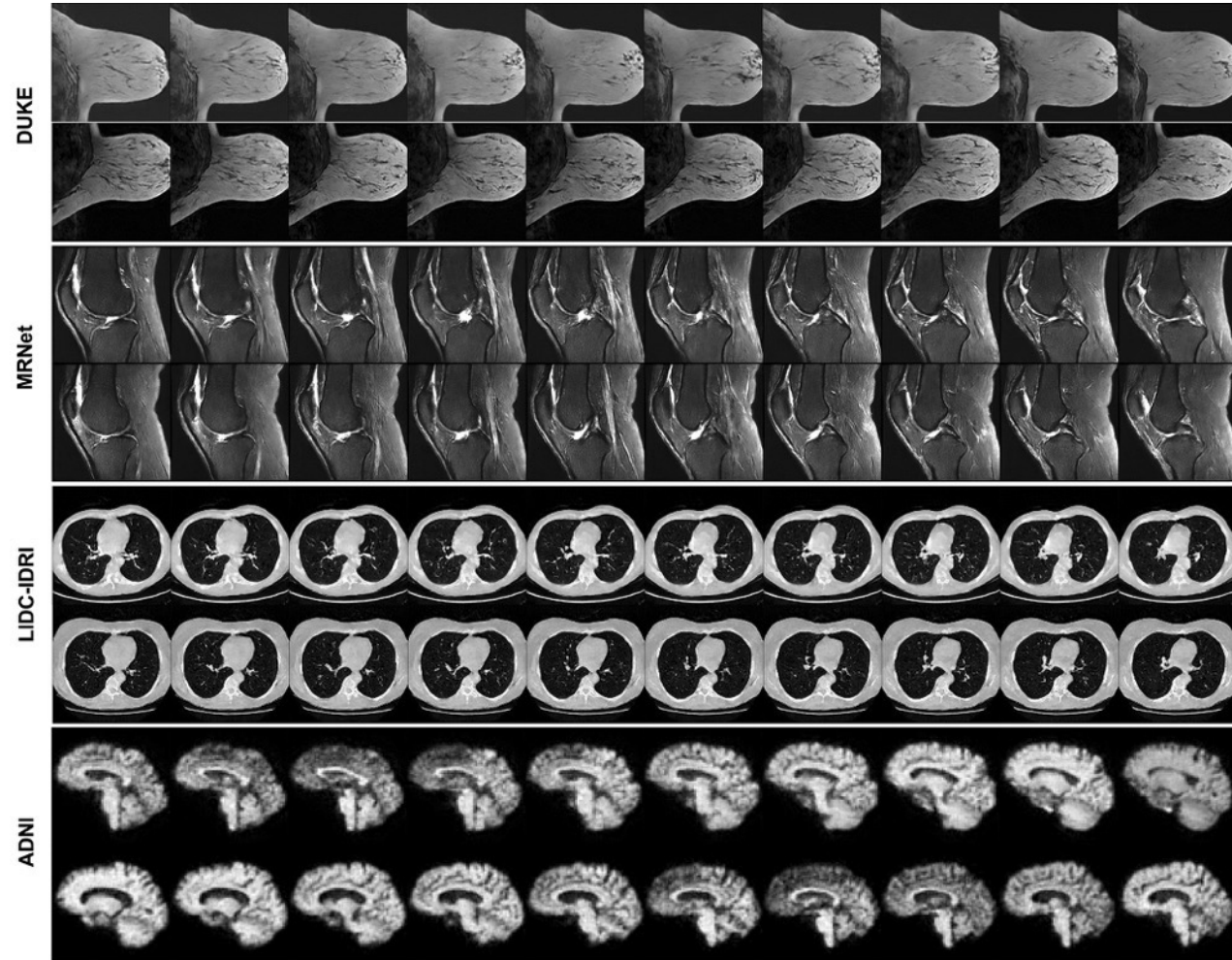
Mazurowski, Maciej A., et al.
"Segment anything model for medical image analysis: an experimental study." *Medical Image Analysis* 89 (2023): 102918.

Generation of Synthetic Medical Images

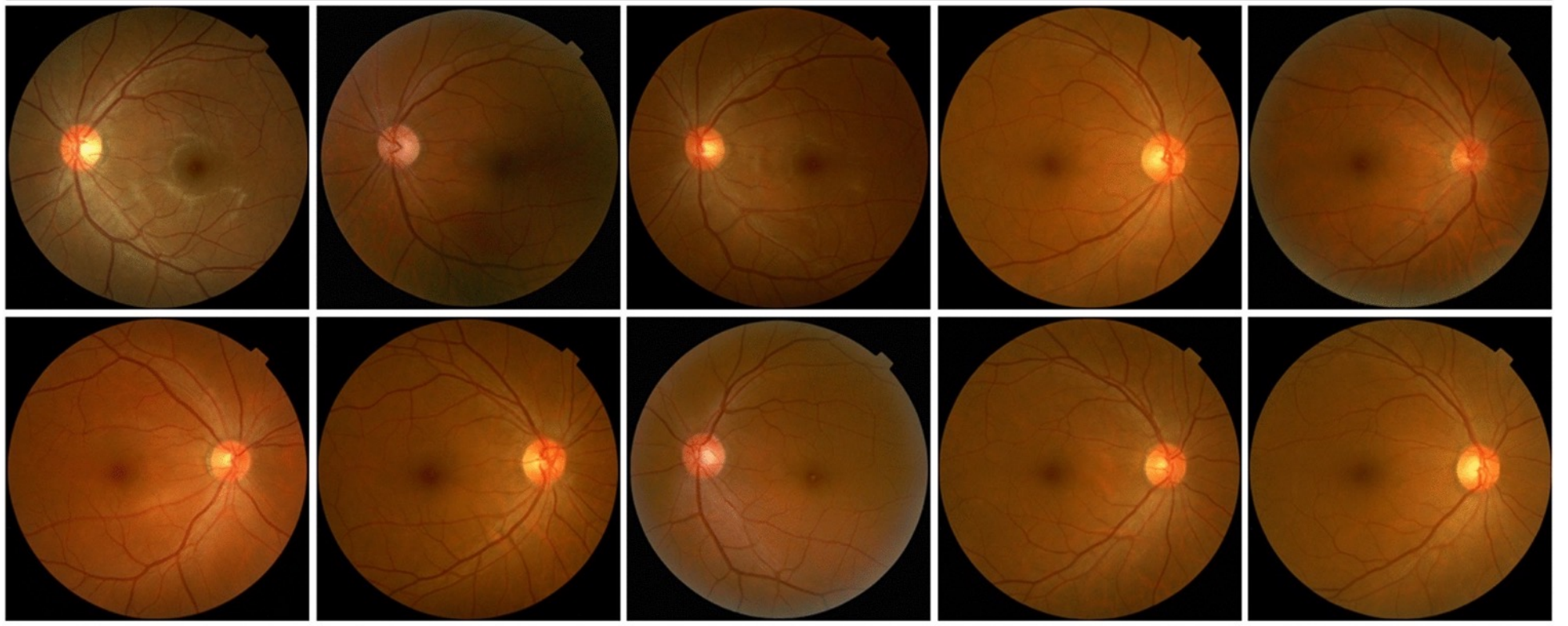
Use cases:

- Privacy / data sharing
- Enrich dataset, e.g. to achieve broader coverage of physiologic variability
- Anomaly detection
- Image denoising

Khader, F., Müller-Franzes, G., Tayebi Arasteh, S. *et al.* Denoising diffusion probabilistic models for 3D medical image generation. *Sci Rep* **13**, 7303 (2023).



Generation of Synthetic Medical Images

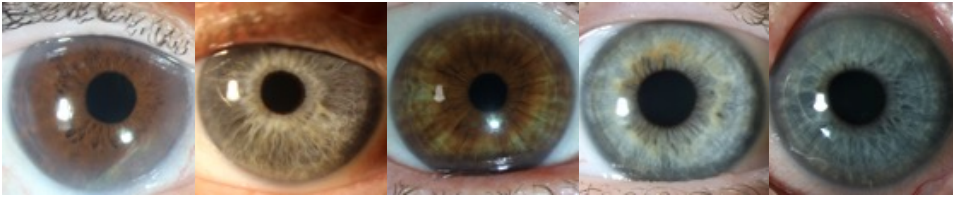


Kim, M., Kim, Y.N., Jang, M. *et al.* Synthesizing realistic high-resolution retina image by style-based generative adversarial network and its utilization. *Sci Rep* **12**, 17307 (2022).

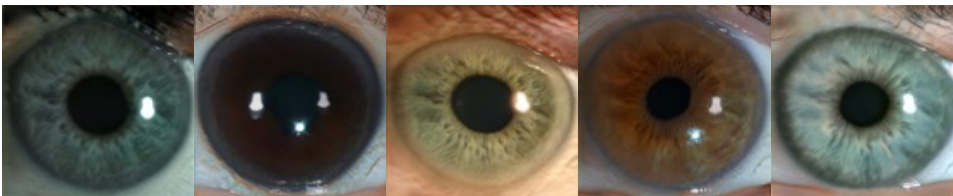
-> **Other Vision Applications**

Often predictive and generative models are complementary.

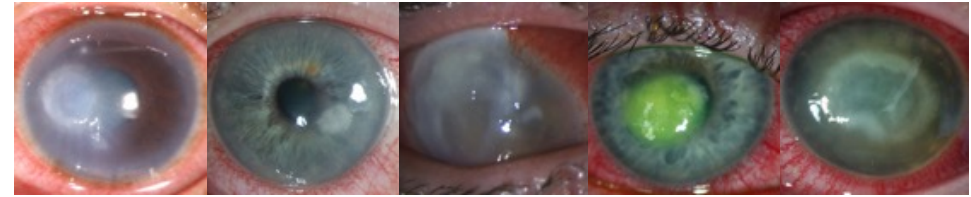
Real Healthy (n=57)



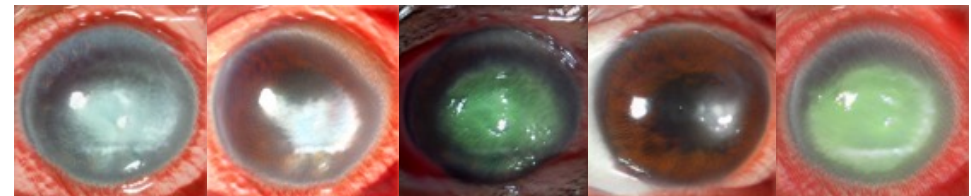
Synthetic Healthy



Real Diseased (n=27)



Synthetic Diseased



Microbial keratitis (MK) detection:

- Learn to generate synthetic data (generative)
- Train an MK detection model (predictive)

What do all have in common? *fine-tuned CNN*

- Predictions are made with a convolutional neural network (CNN)*, which is a neural network with *convolutional* (rather than *fully-connected*) layers.
- These models are not trained from scratch. Instead of beginning with random parameters, we begin with parameters learned from a different, usually non-medical task.
 - Why is this a good idea?
 - How can we develop our own CNN model?

*rarely, a vision transformer; more on this later

How do CNNs work?

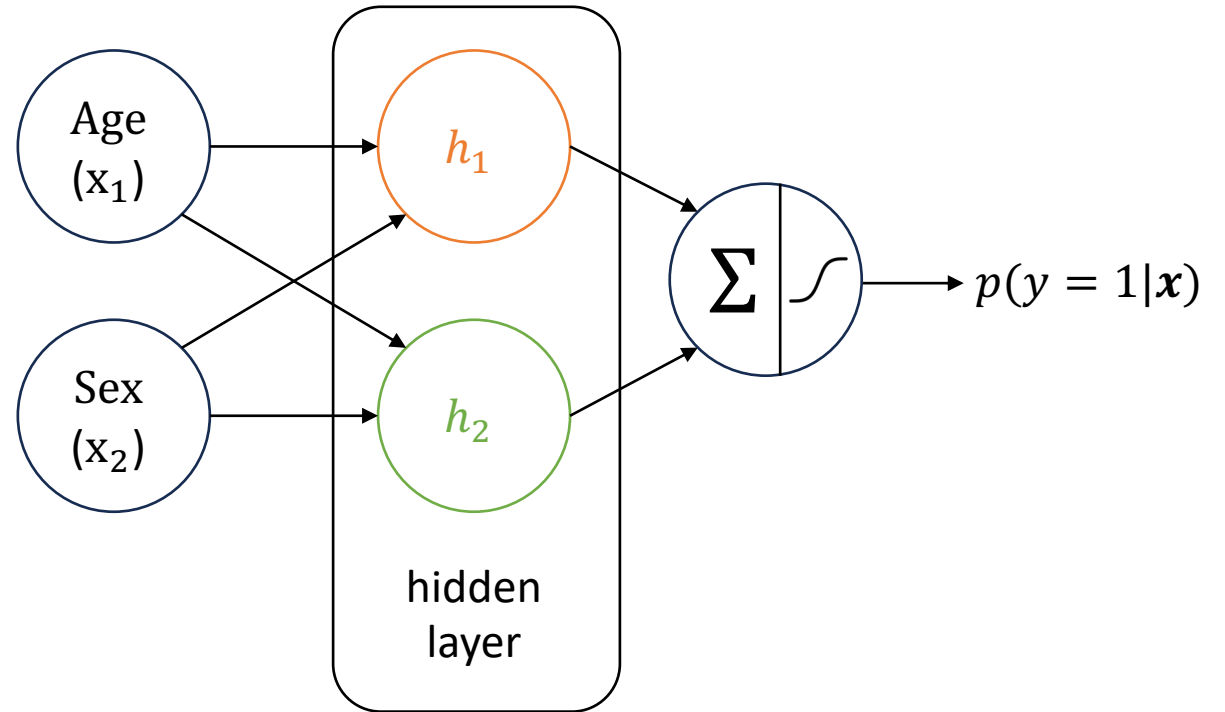
Two key ideas:

- 1) Hierarchical extraction of visual features (like other deep NNs)
- 2) Translation invariance

Hierarchical feature extraction

Recall our simple fully-connected NN

- Neuron 1 (h_1):
 - $h_1 = 1$ if (female ≤ 60)
 - $h_1 = 0$ otherwise
- Neuron 2 (h_2):
 - $h_2 = 1$ if (male > 60)
 - $h_2 = 0$ otherwise

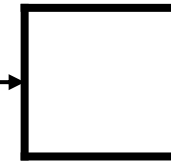


Detect *complex, nonlinear* features by first identifying simpler features.

The relationship between individual pixels and the outcome is not linear.



x , data/features for
a subject or patient



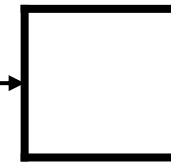
y , associated
value or label

End goal: predict y from x

The relationship between individual pixels and the outcome is not linear.



x , data/features for
a subject or patient



y , associated
value or label

End goal: predict y from x

We must build a hierarchy of features.



pixels

low-level
motifs

- edges
- shapes
- textures

high-level motifs are
specific arrangements
of low-level motifs

high-level
motifs

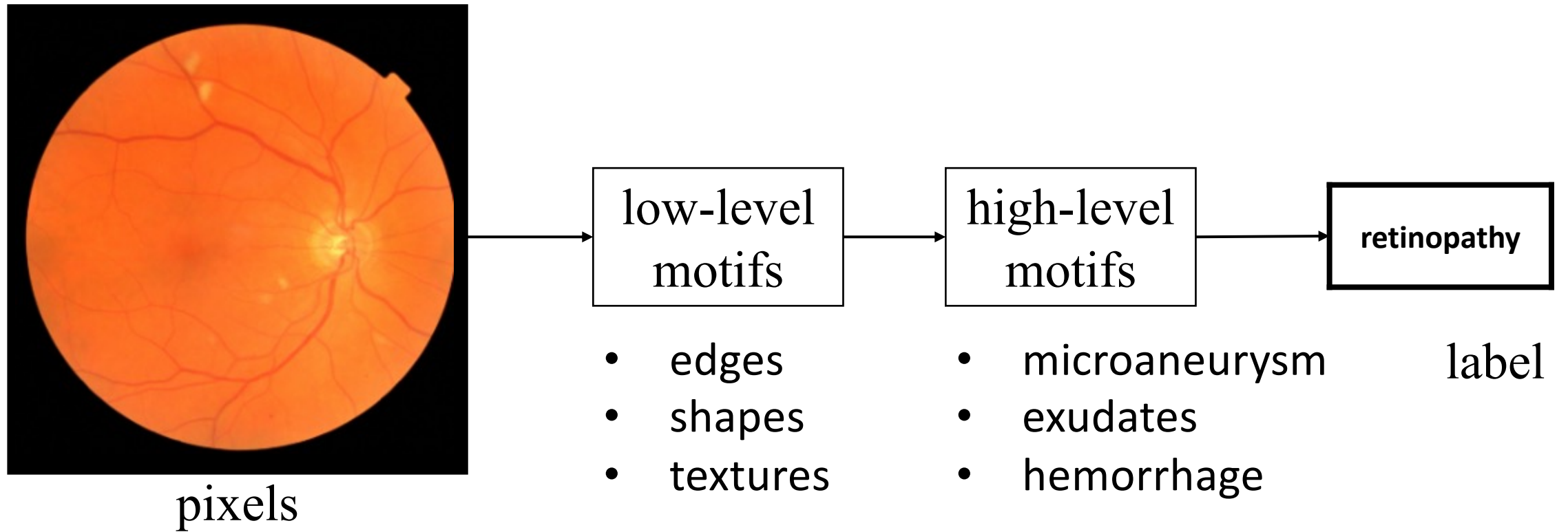
- eyes
- ears
- paws

dog

label

End goal: predict *dog* from *pixels*

We must build a hierarchy of features.



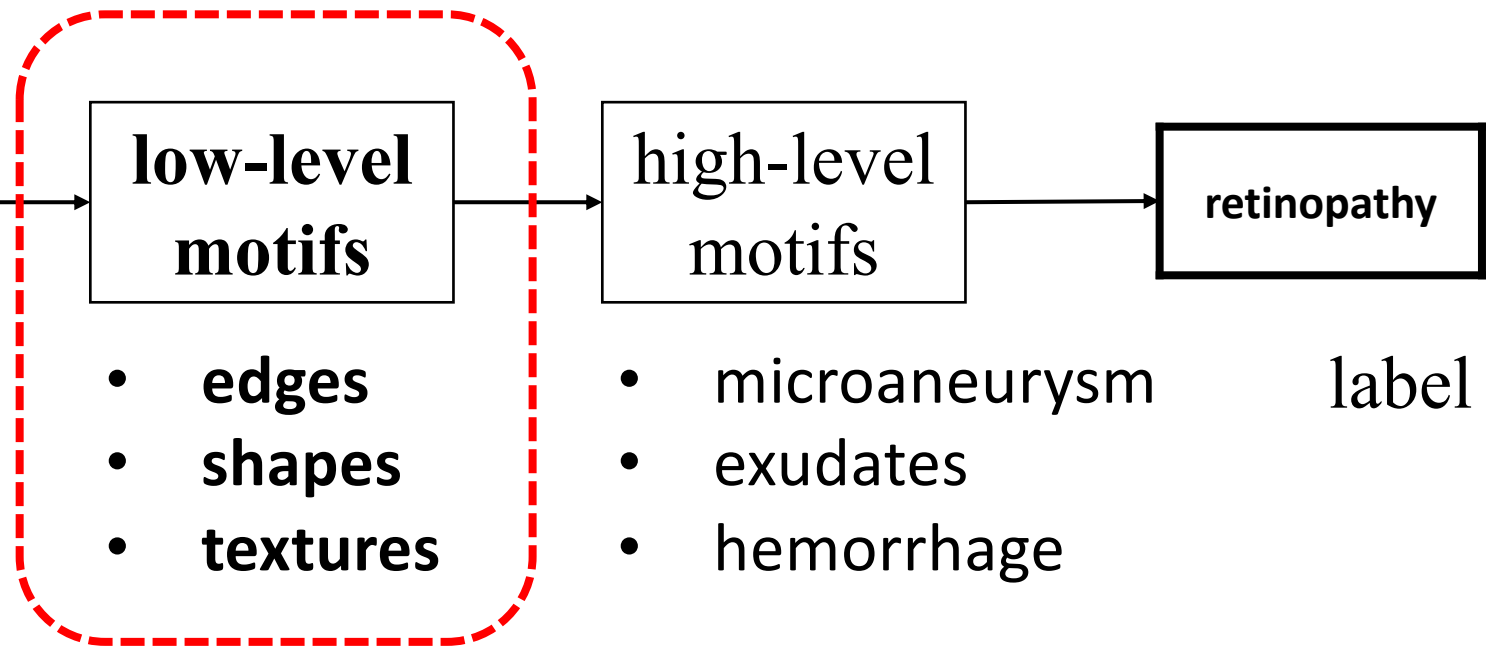
End goal: predict *retinopathy* from *pixels*

We must build a hierarchy of features.



pixels

low-level motifs are common to *all* naturalistic images...



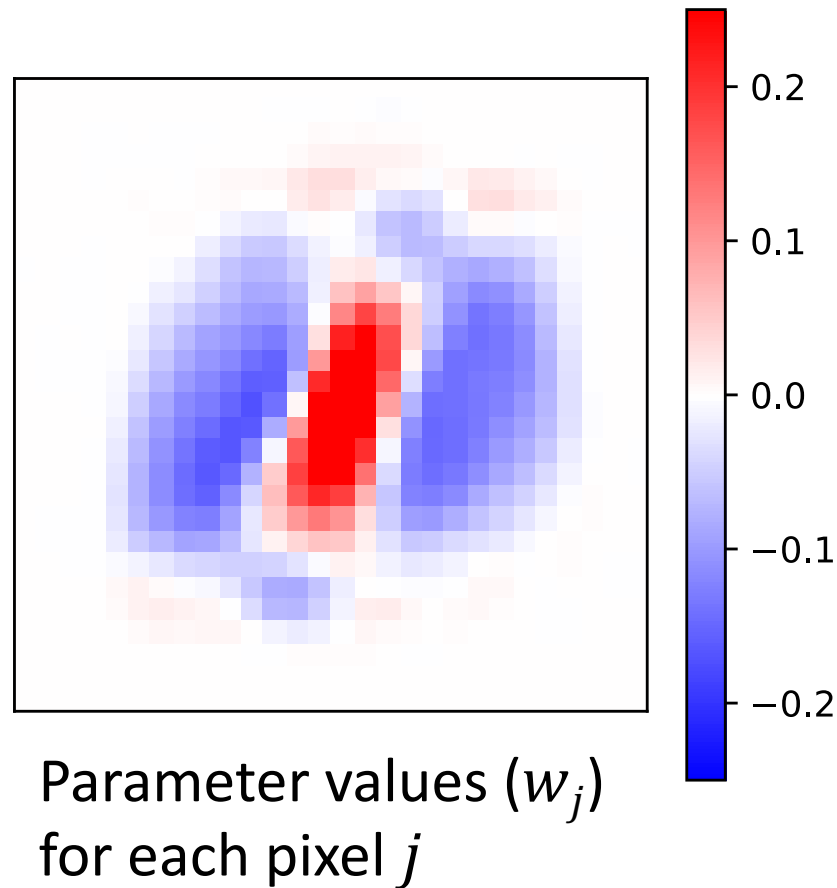
End goal: predict *retinopathy* from *pixels*

Why start with a pre-trained model?

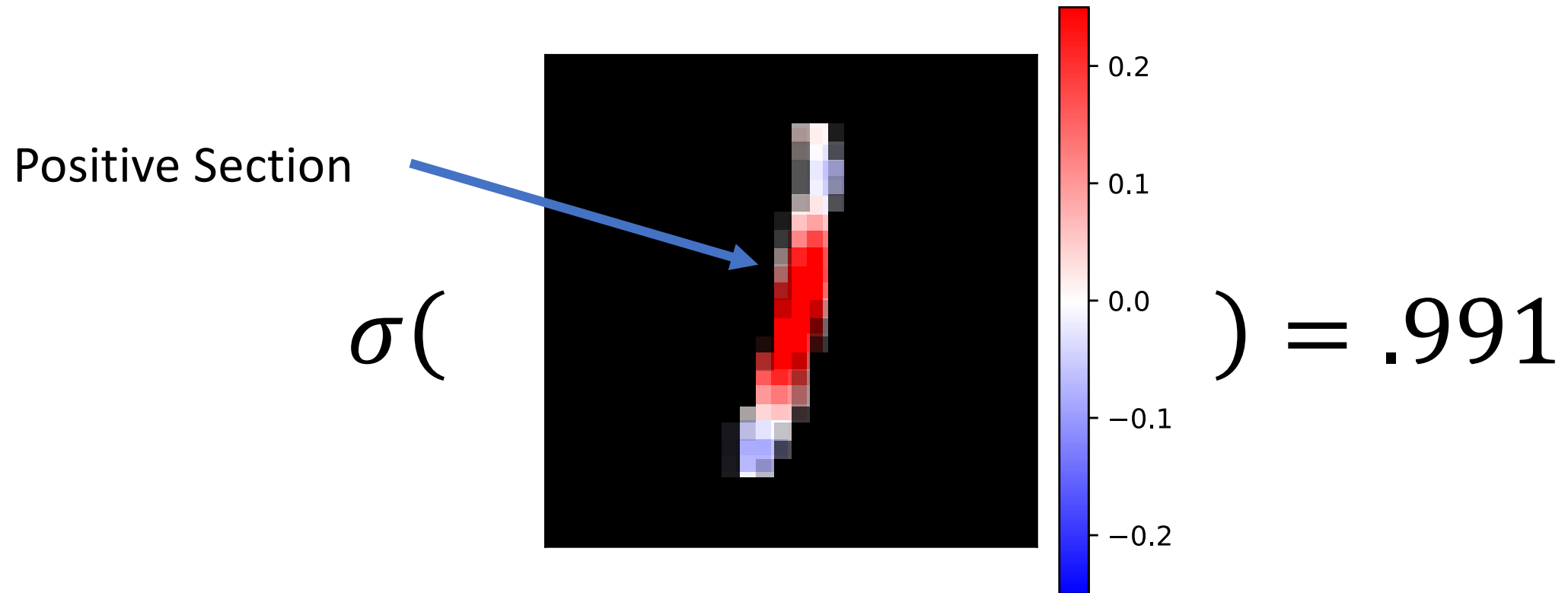
- Instead of beginning with random parameters, we begin with parameters learned from a different, usually non-medical task.
- This is because these parameters are *better than random*: many features that are helpful in classifying naturalistic images are also helpful in classifying medical images.
- This is particularly true *early* in the feature hierarchy (i.e. low-level motifs; early layers).

Translation invariance

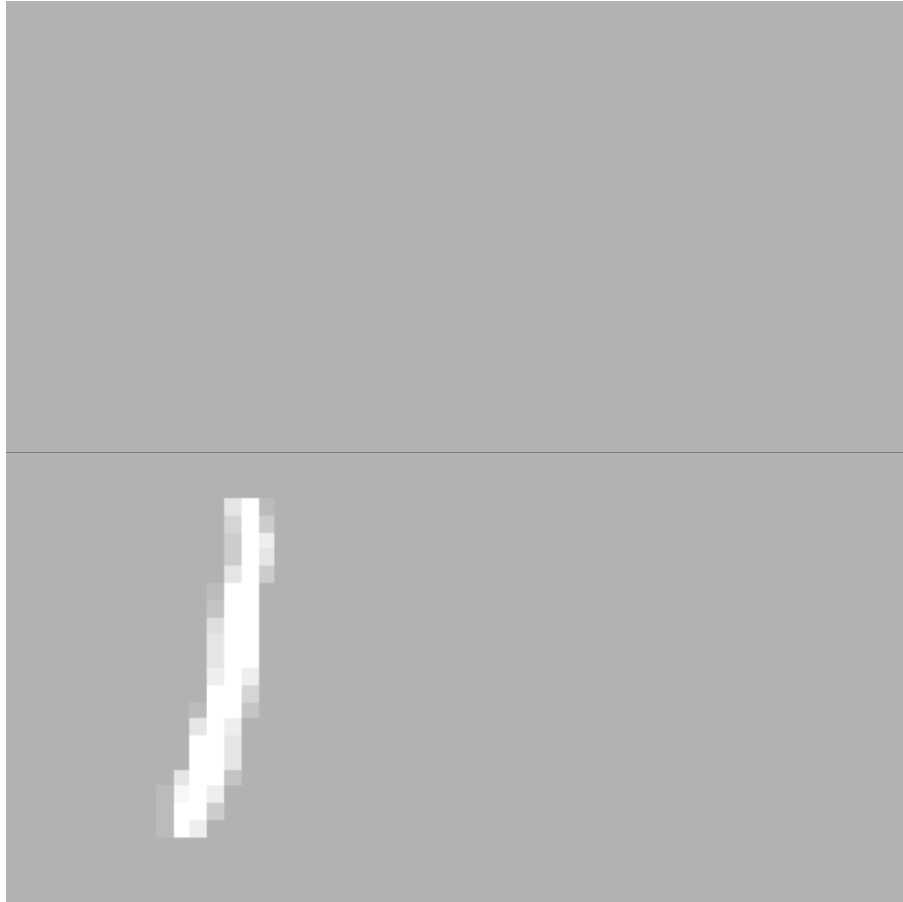
Recall our filter for detecting ones



A “Filter” to Detect Ones

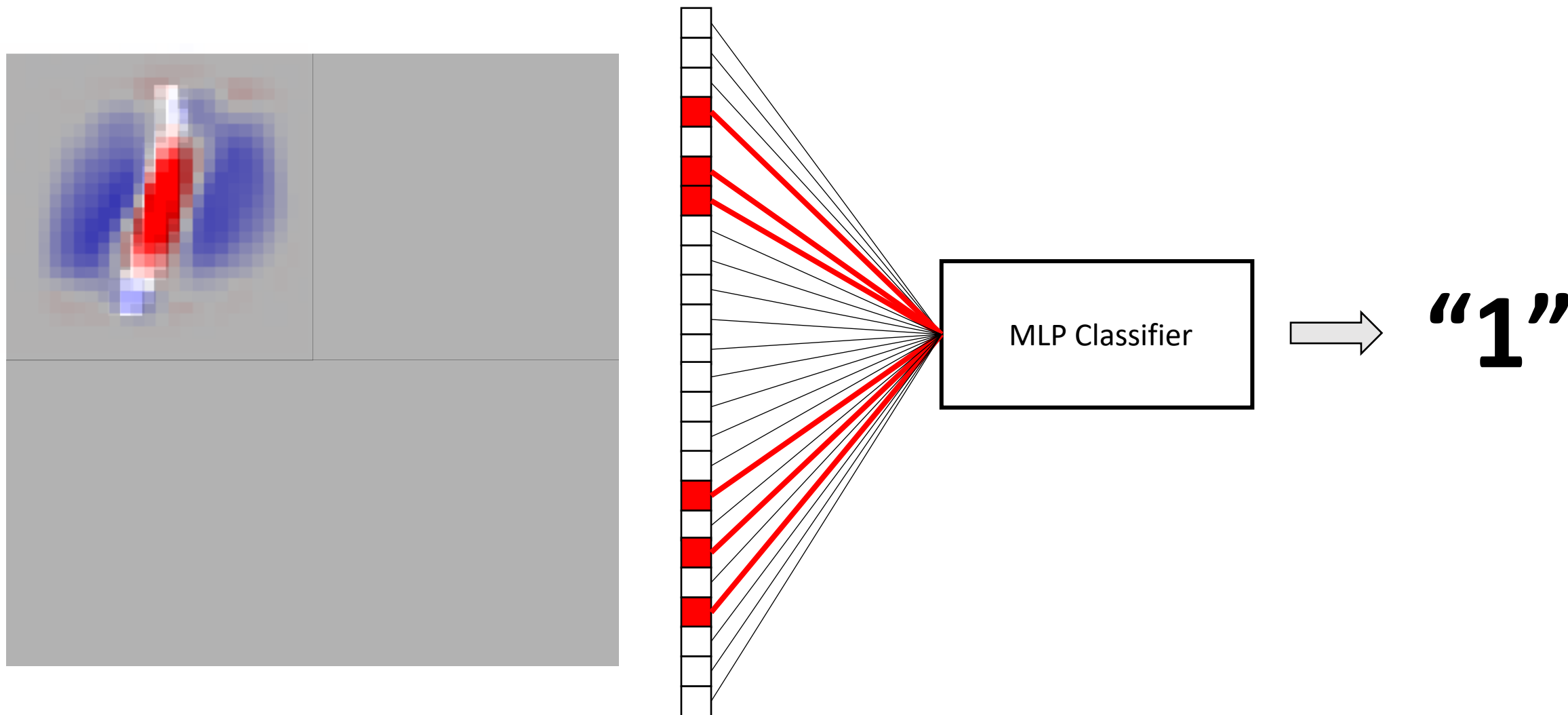


We'd like to be able to find a 1 *anywhere*

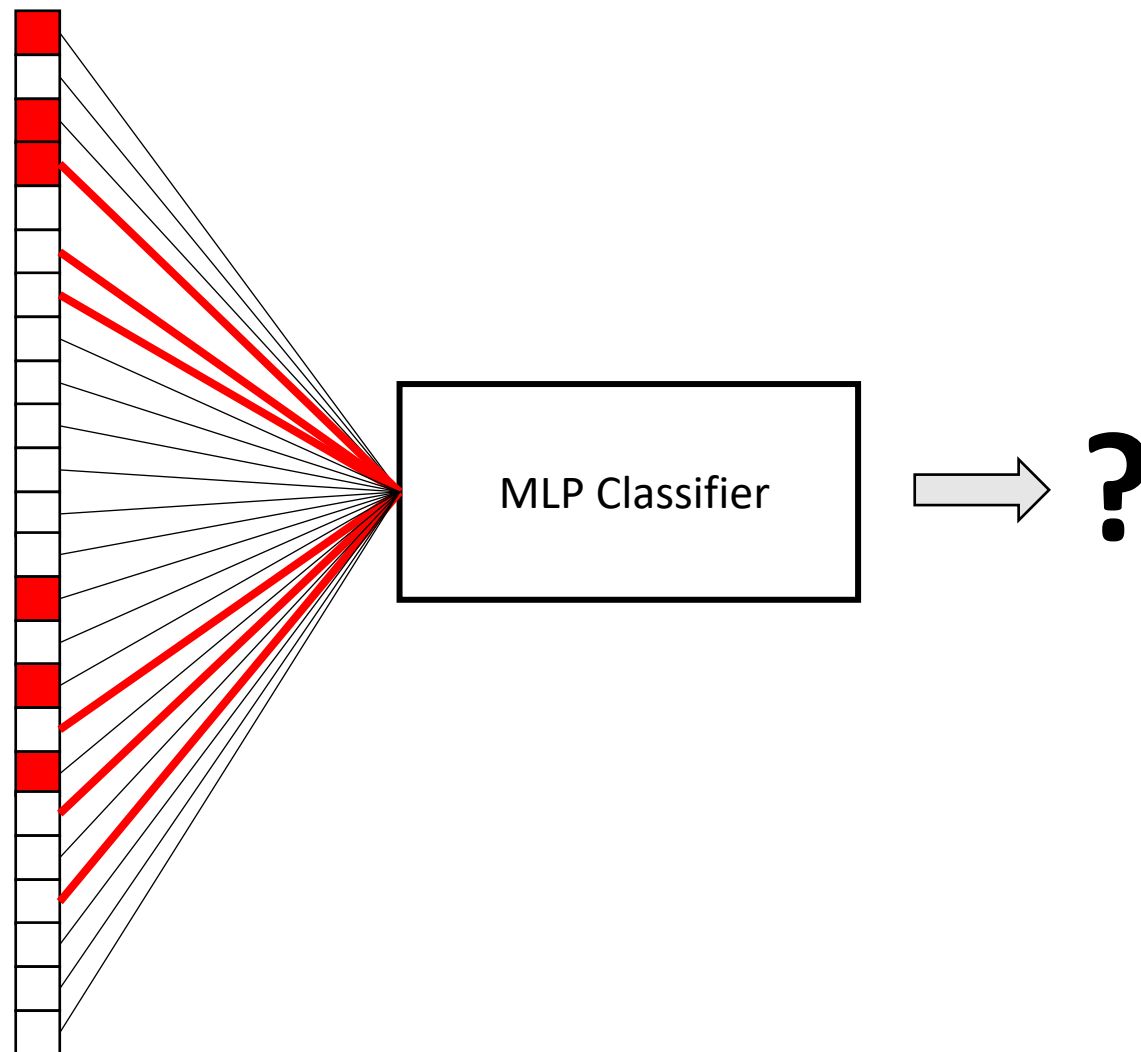
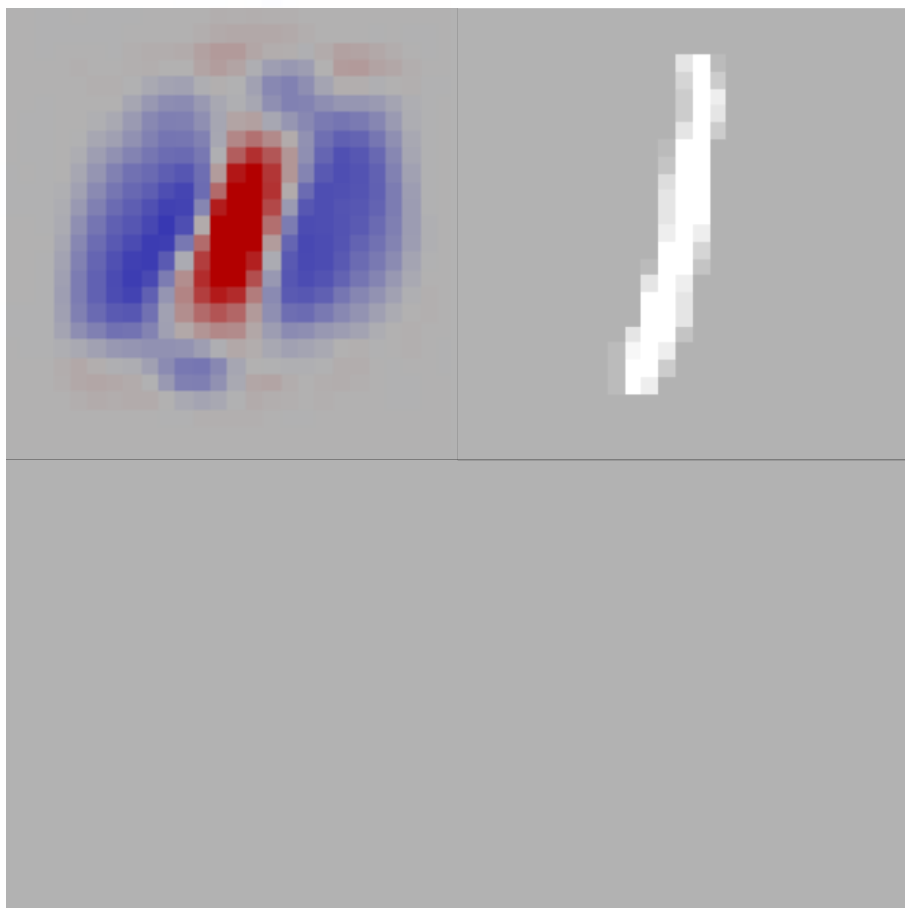


- Images are large
- Whole-image filters have many parameters
- We want to learn a hierarchy of increasingly complex features
- (key motivation) **in images, features are spatially cohesive**
- Consider how to make an efficient network in this context

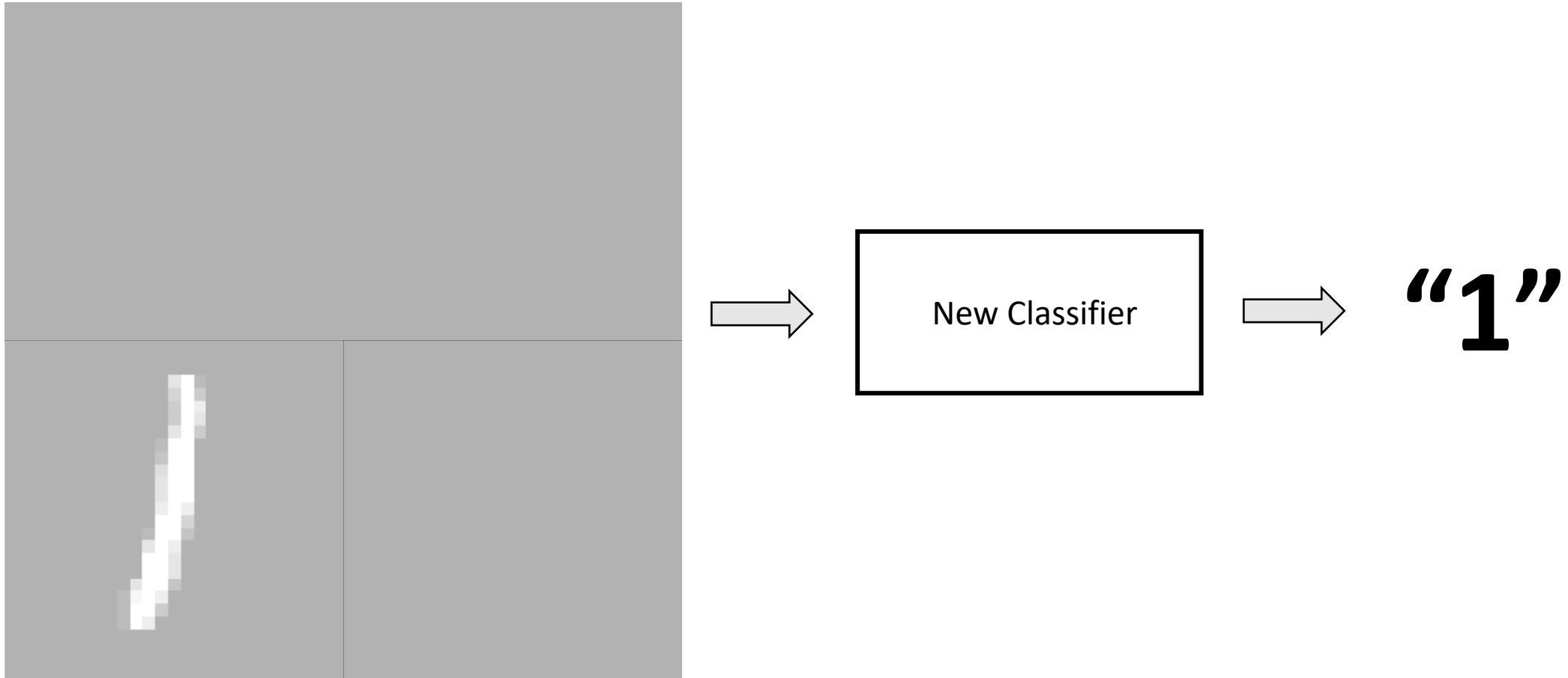
Our previous approach looks for a 1 at a specific location.



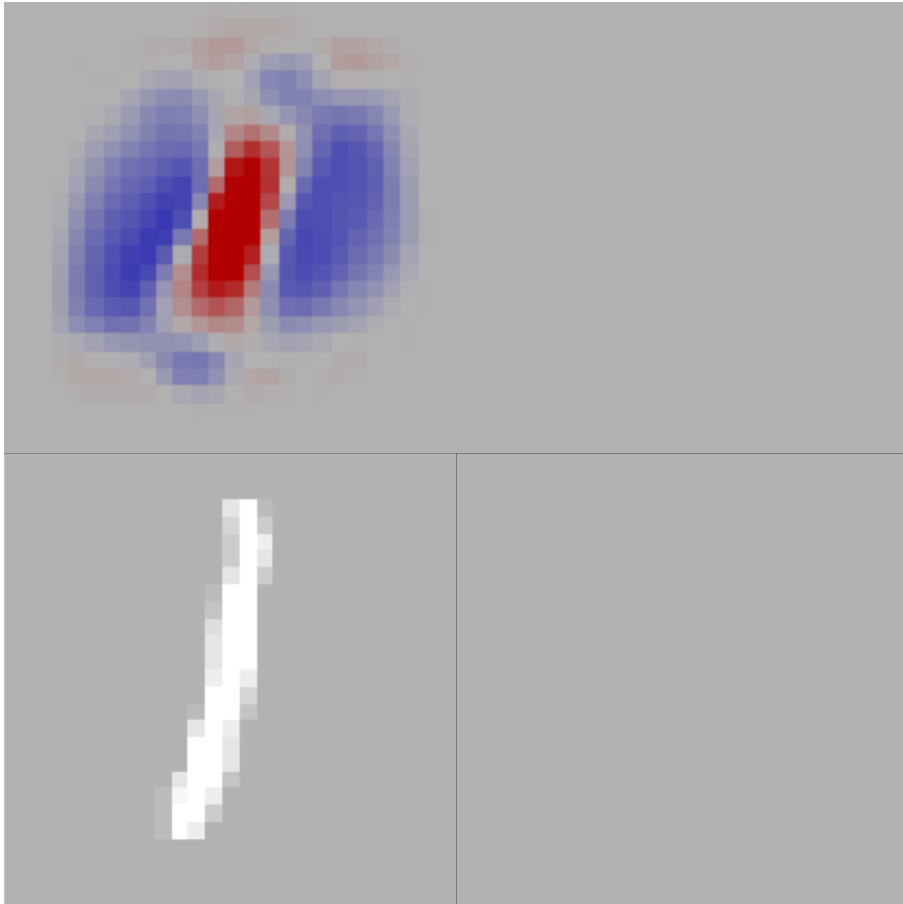
If we move the position of the 1, it no longer works.



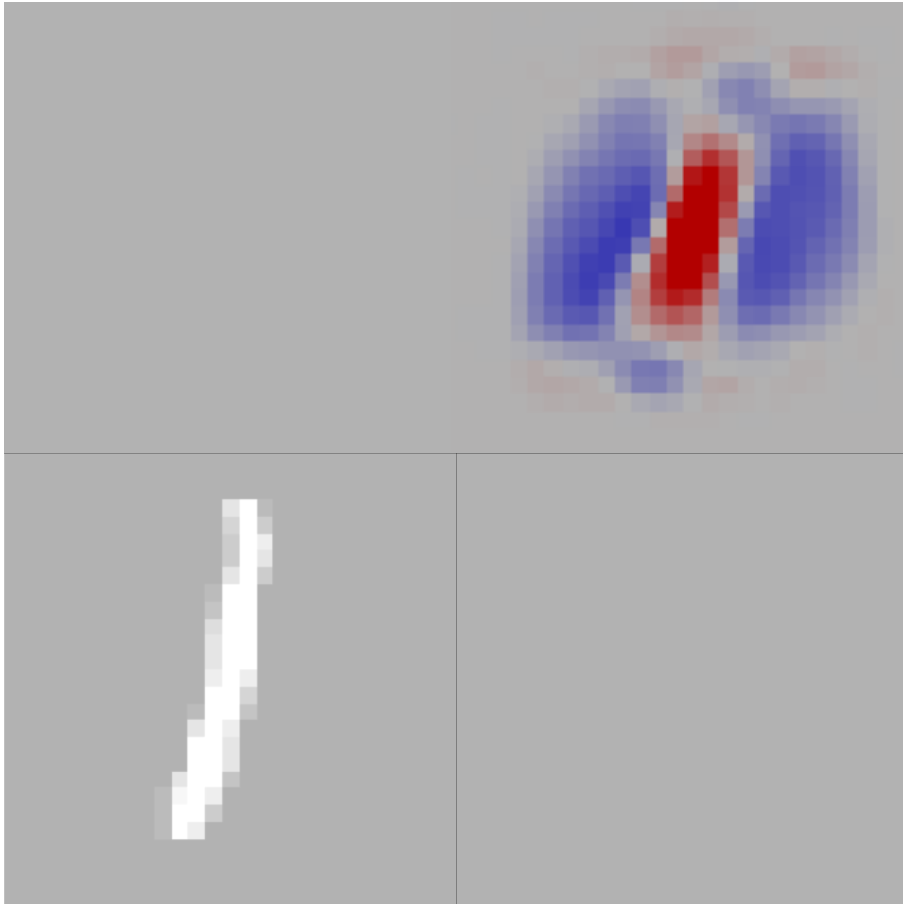
Instead of using logistic regression or an MLP, let's look for a new kind of model, one with more flexible filters



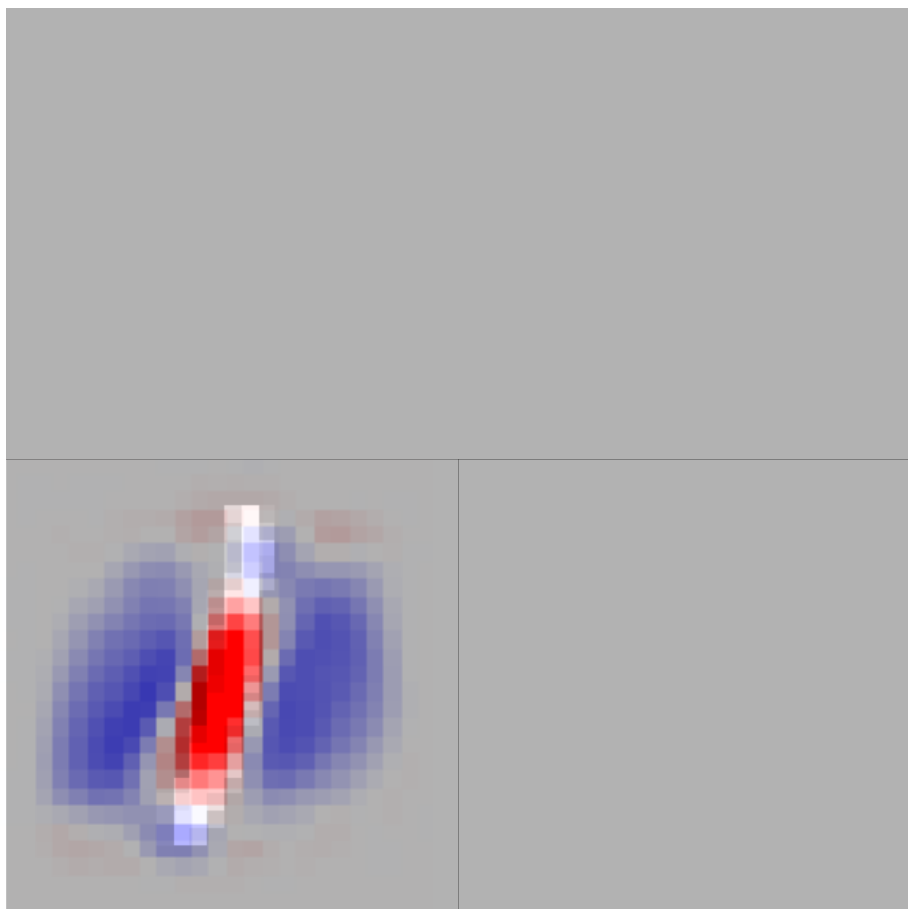
Instead of a filter that's the size of the whole image, we'd like a smaller filter that we can move around



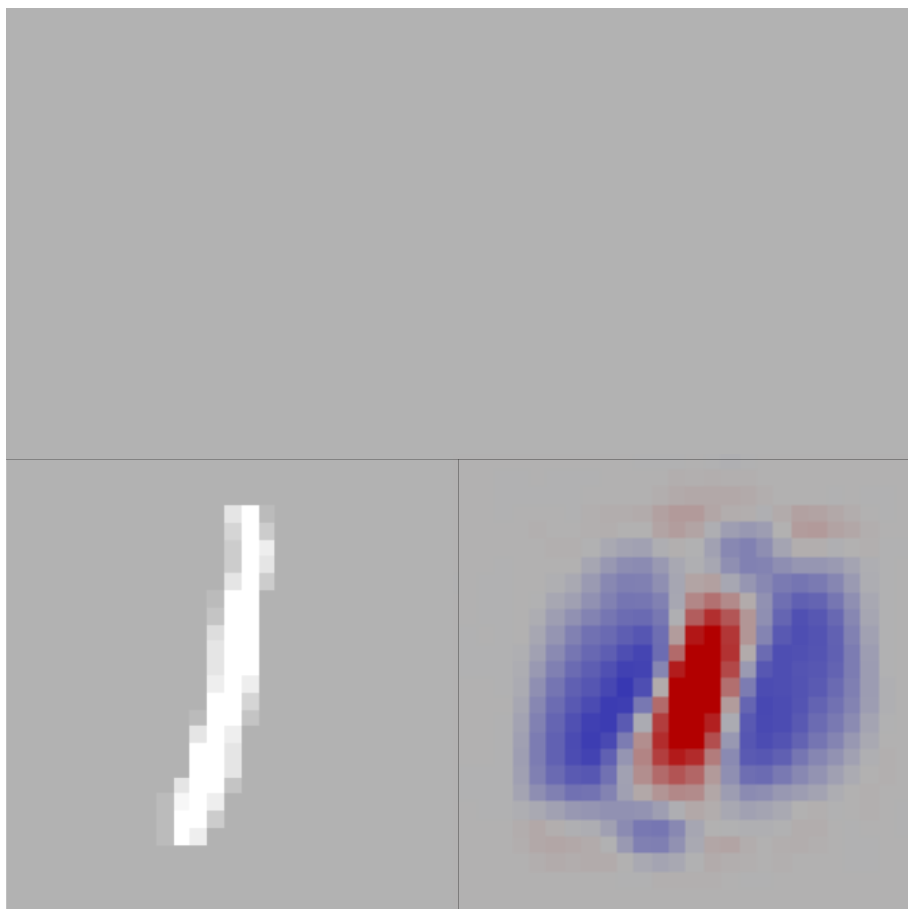
As we move this filter, we calculate the inner product between the filter itself and the portion of the image that's underneath it.



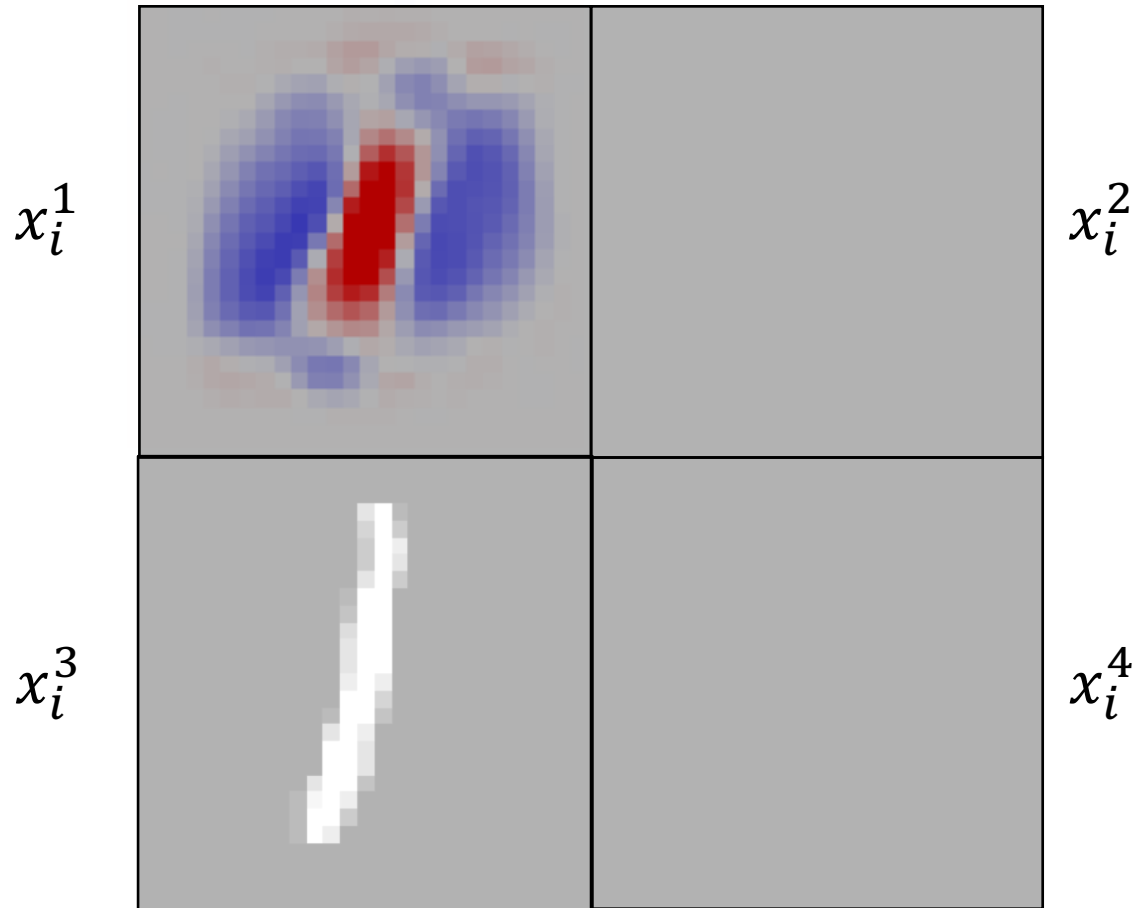
When the filter is placed over a region that looks like the filter, the inner product (i.e. filter output) will be large.



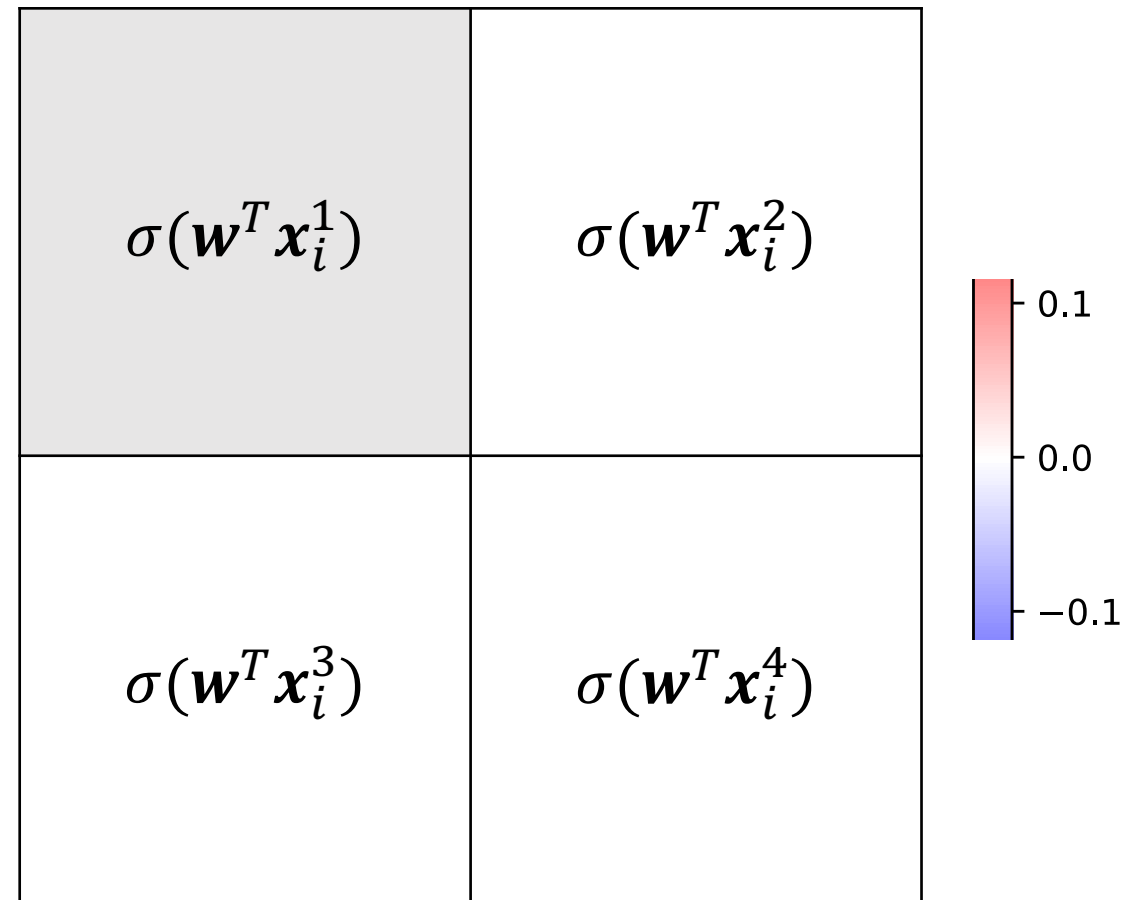
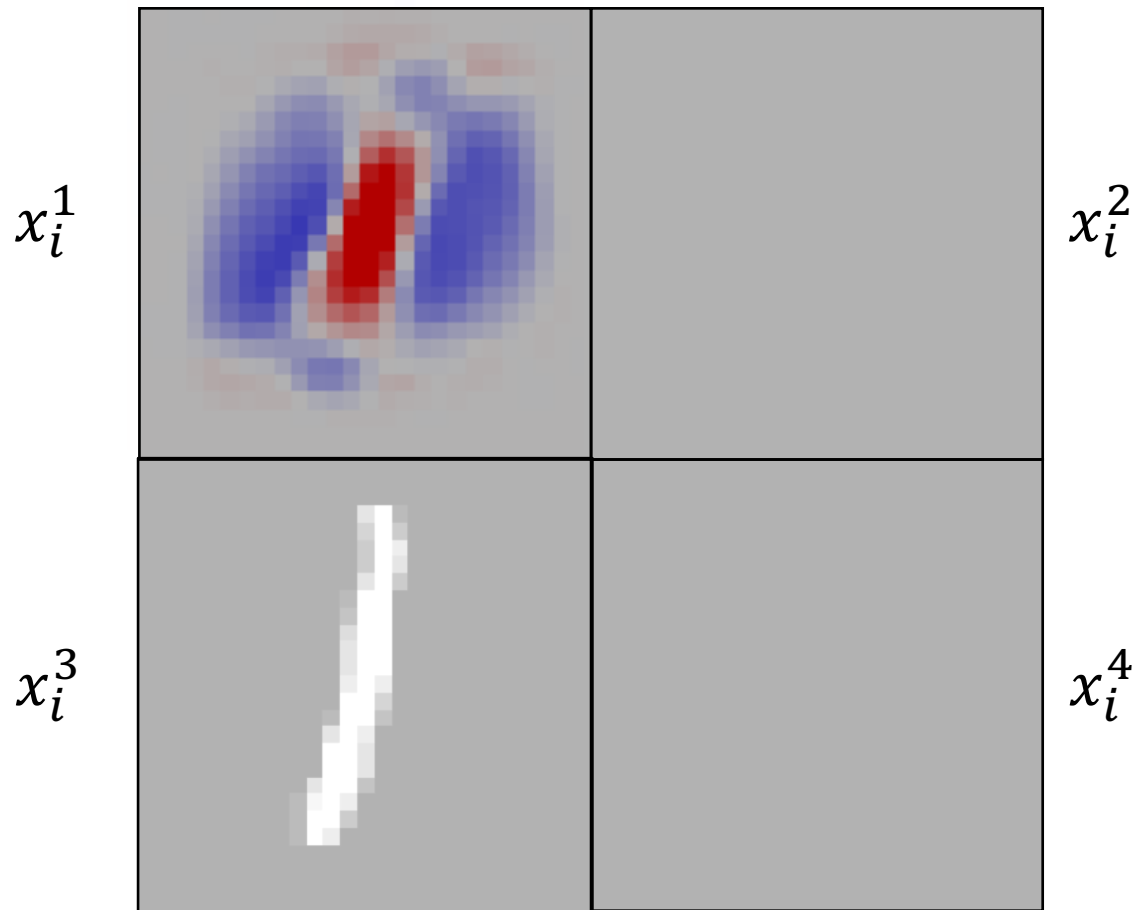
When it's placed over a region that does not look like the filter, the inner product (i.e. filter output) will be small



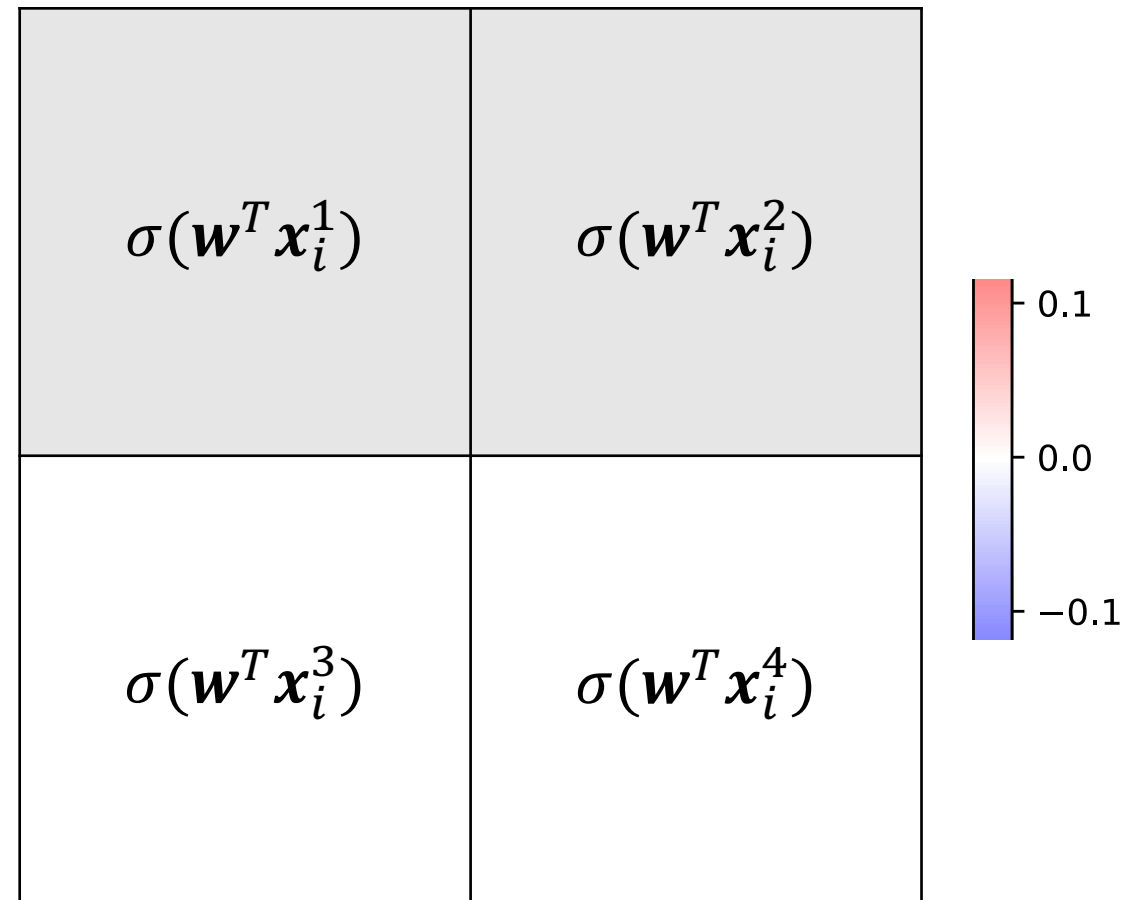
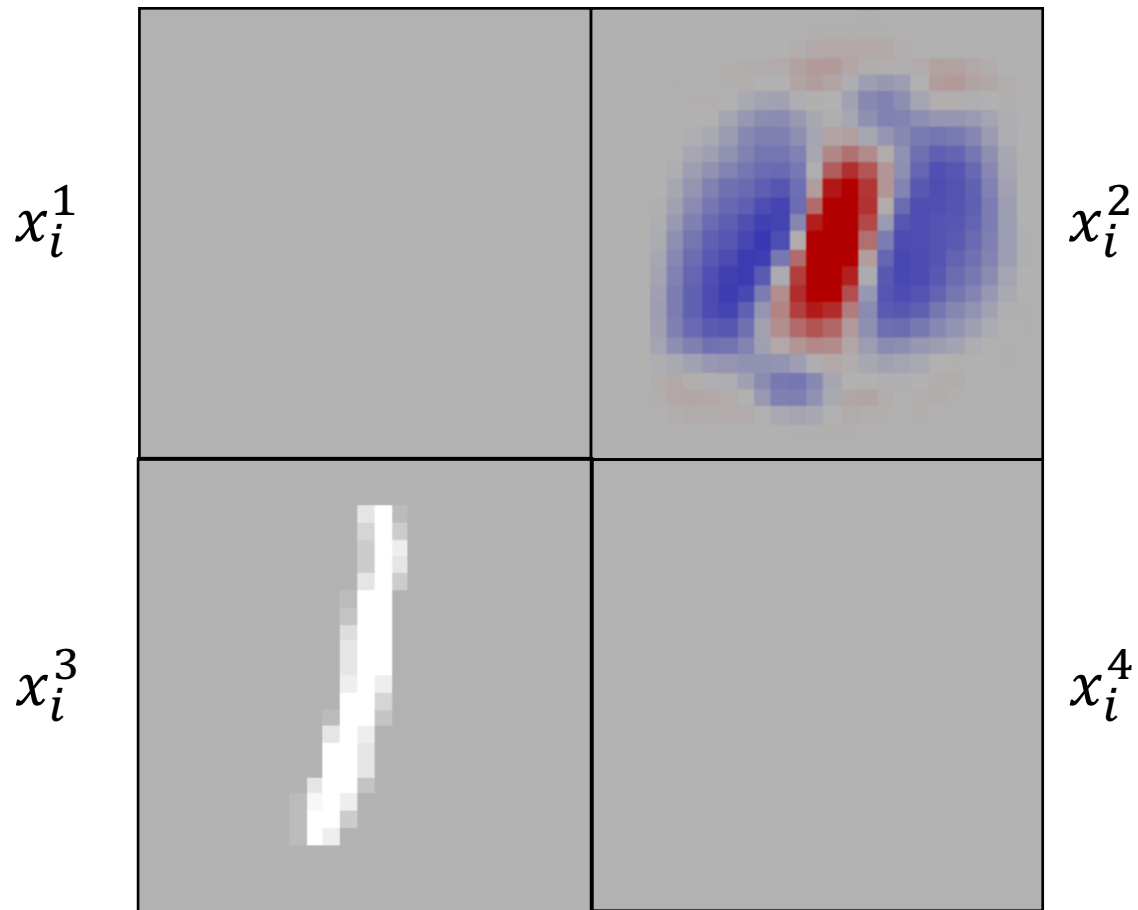
Examining filter output $\sigma(\mathbf{w}^T \mathbf{x}_i^R)$, where $\mathbf{x}_i^R$ is the region of image i where the filter is placed.



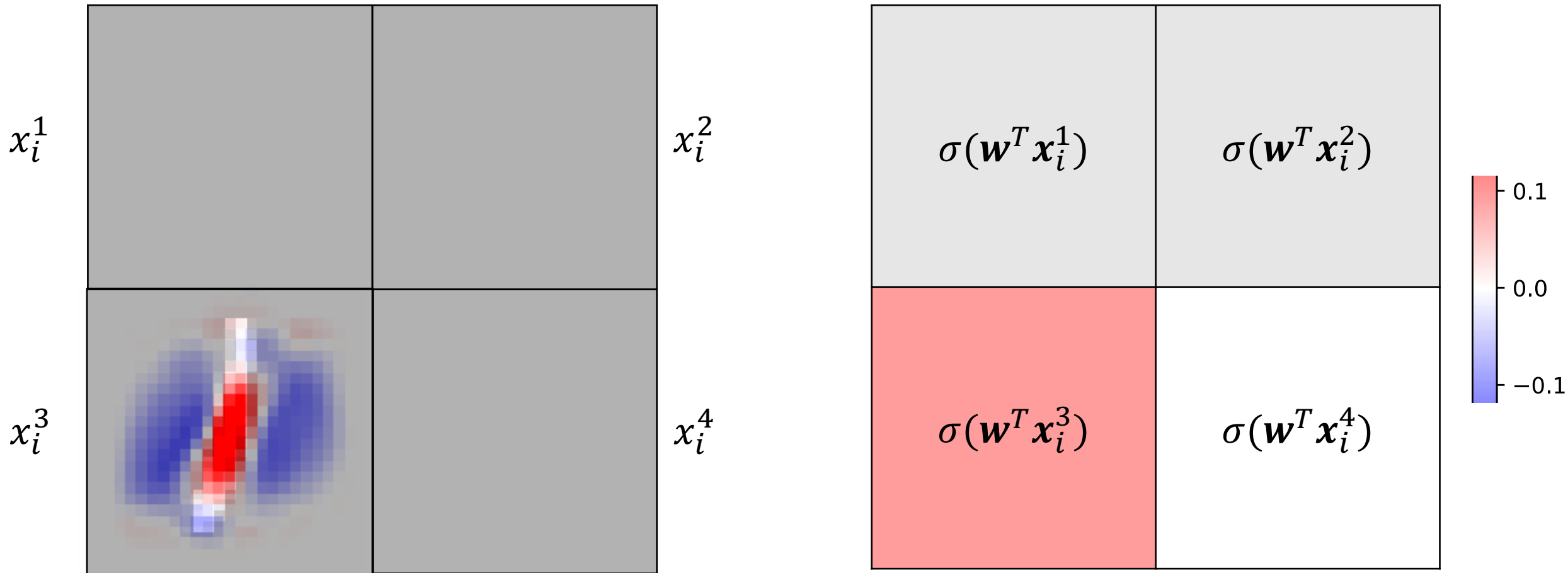
Examining filter output $\sigma(\mathbf{w}^T \mathbf{x}_i^R)$, where $\mathbf{x}_i^R$ is the region of image i where the filter is placed.



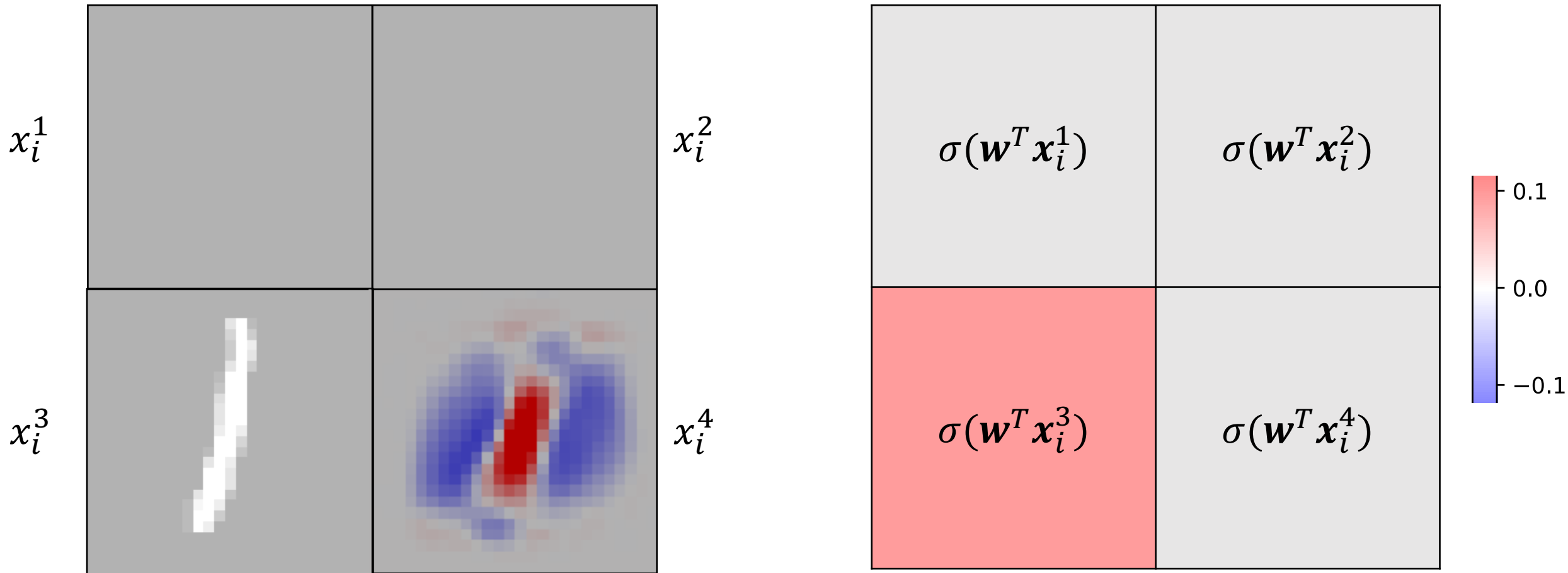
Examining filter output $\sigma(\mathbf{w}^T \mathbf{x}_i^R)$, where $\mathbf{x}_i^R$ is the region of image i where the filter is placed.



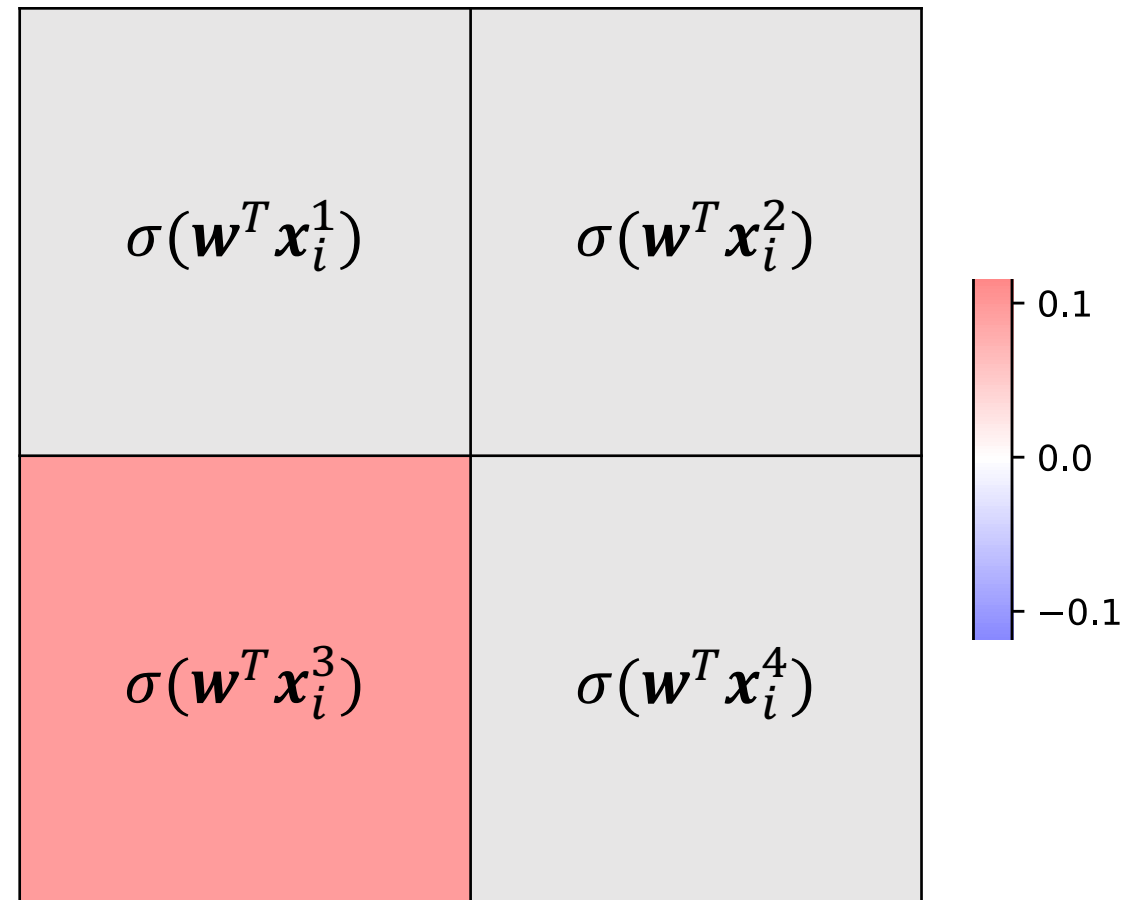
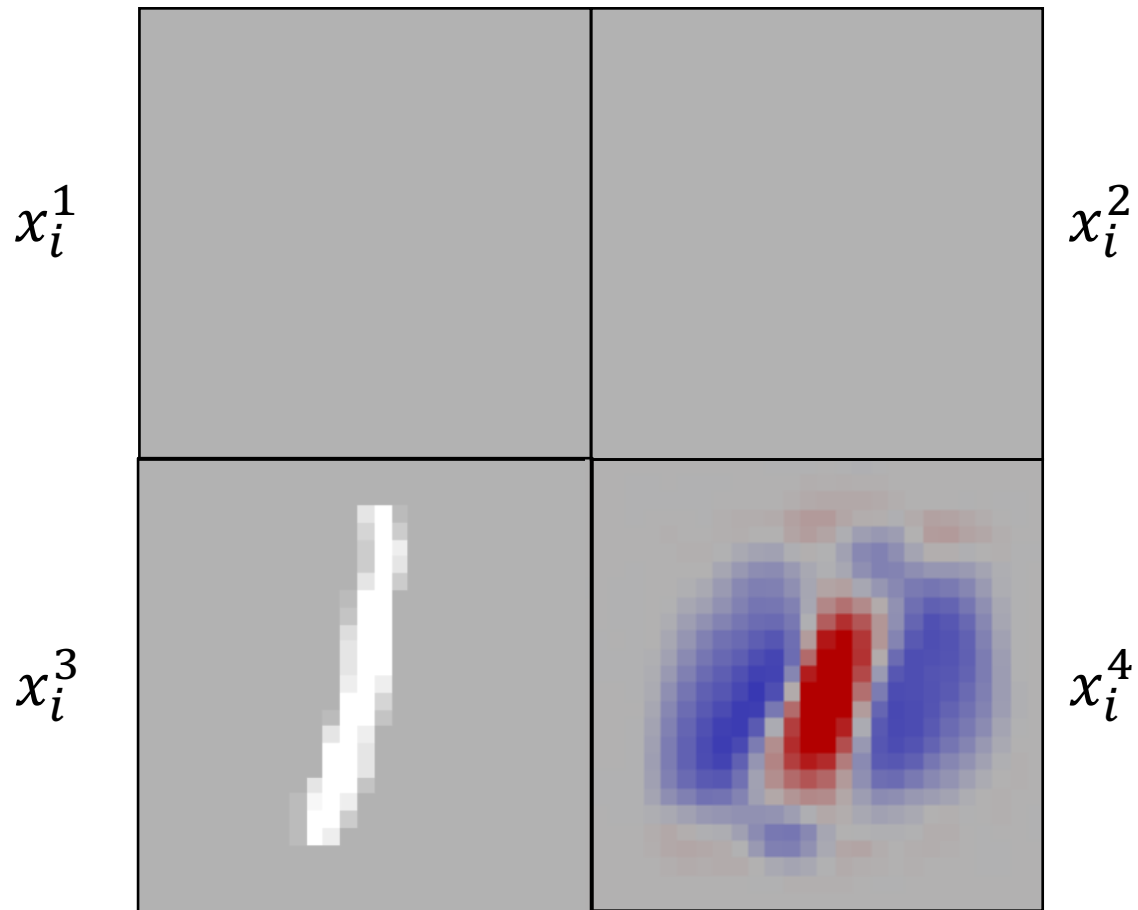
Examining filter output $\sigma(\mathbf{w}^T \mathbf{x}_i^R)$, where $\mathbf{x}_i^R$ is the region of image i where the filter is placed.



Examining filter output $\sigma(\mathbf{w}^T \mathbf{x}_i^R)$, where $\mathbf{x}_i^R$ is the region of image i where the filter is placed.



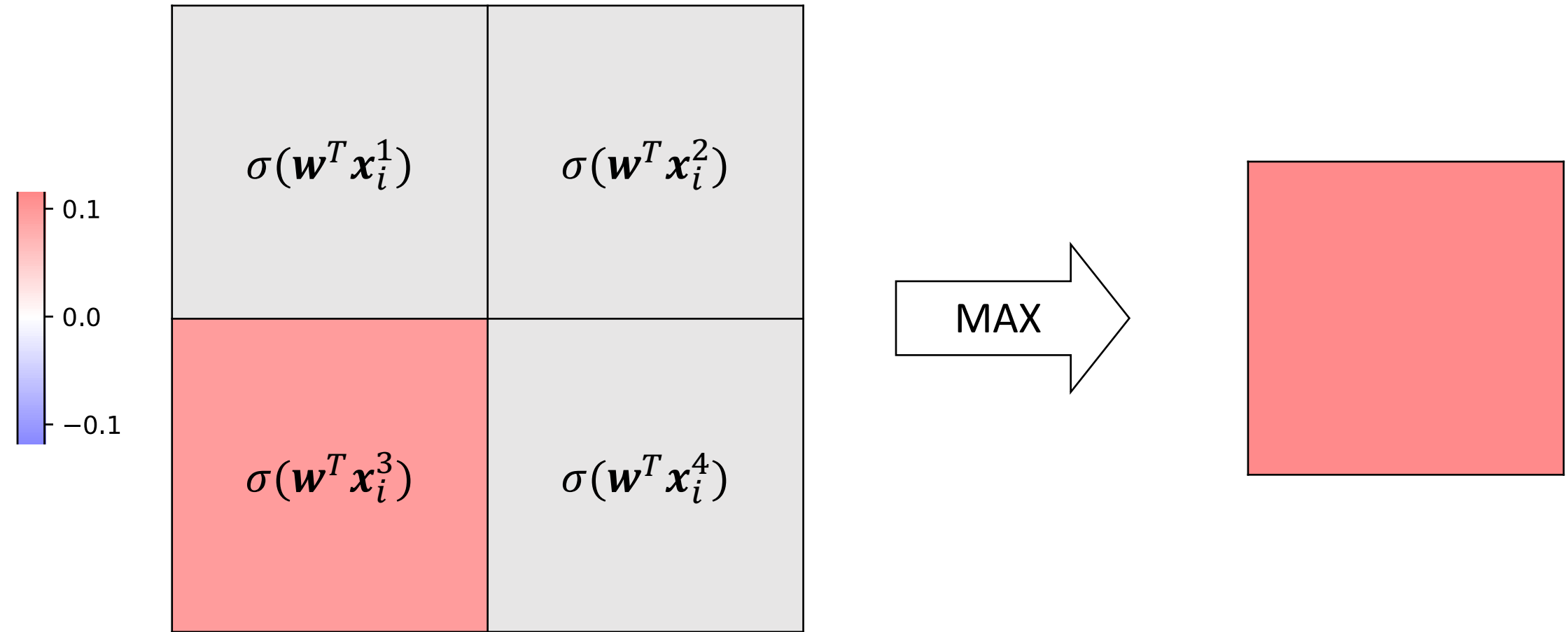
We'd like to be able to find a 1 *anywhere*



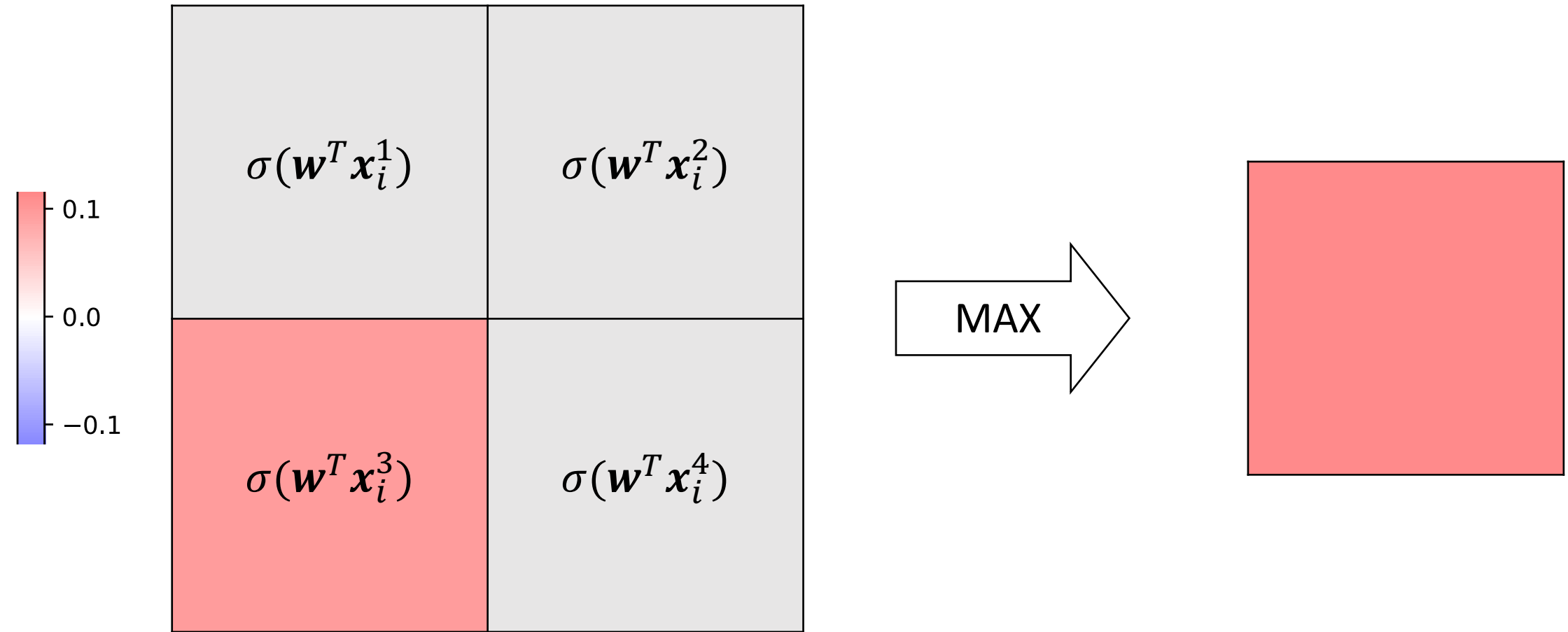
Apply convolutional filter -> obtain feature map



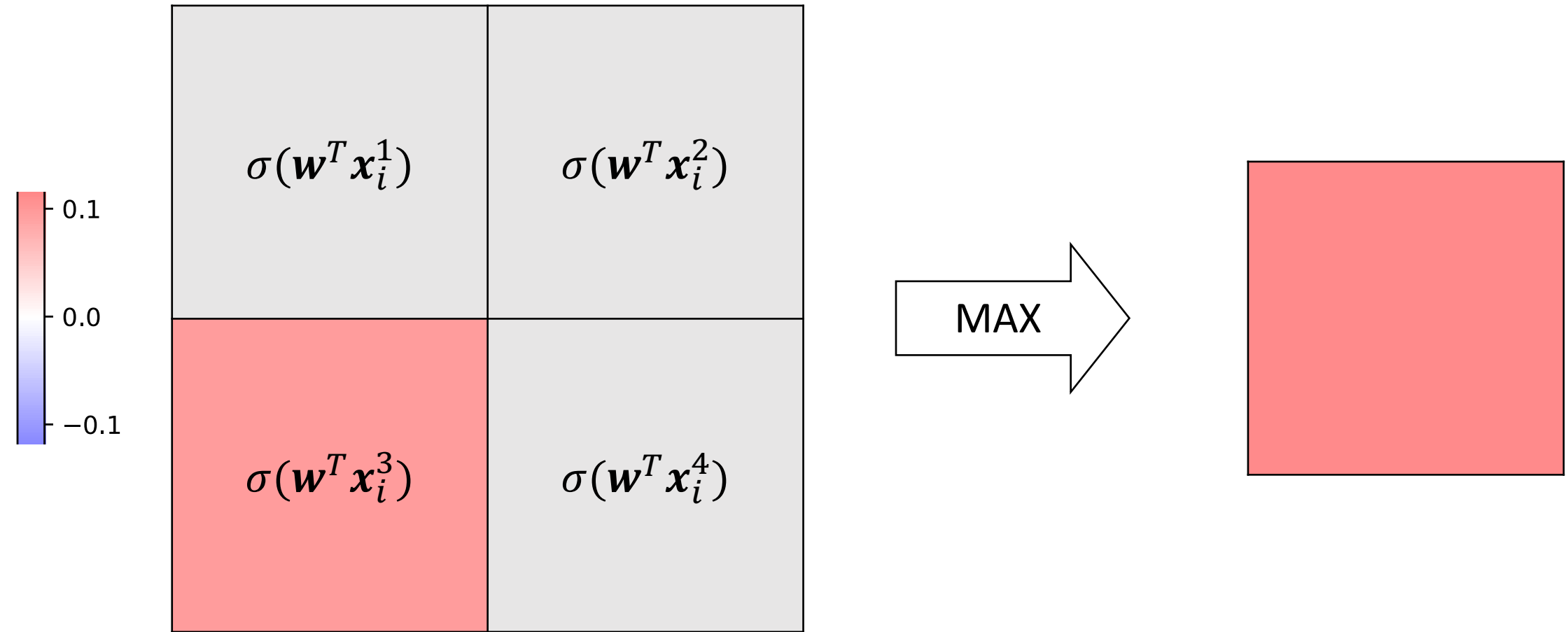
We'd like to be able to find a 1 *anywhere*



This idea is called “max pooling”, and is widely used in CNNs to determine whether features are present in a given region

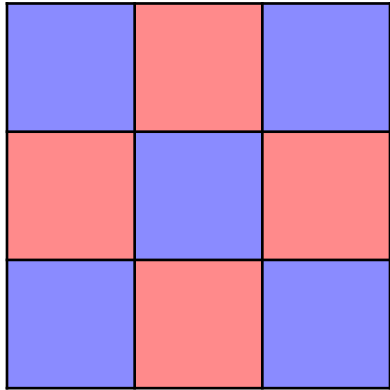


Spatial convolution followed by max pooling is translation invariant.
(discuss: any caveats?)

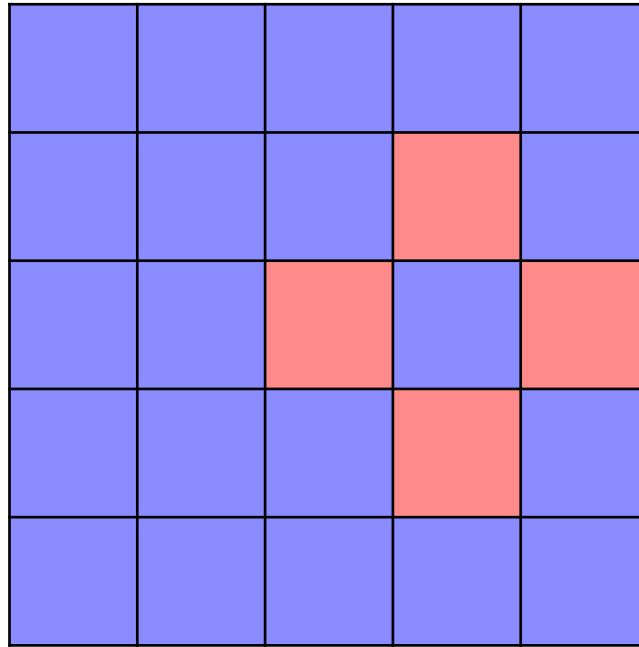


Spatial convolution example

An Example...



filter



image

An Example...

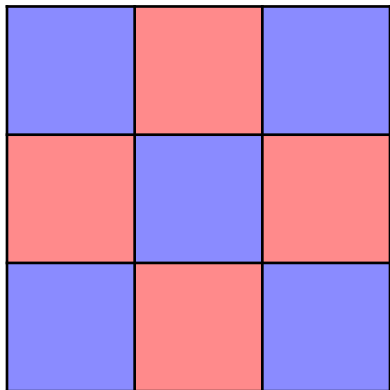
-1	1	-1
1	-1	1
-1	1	-1

filter

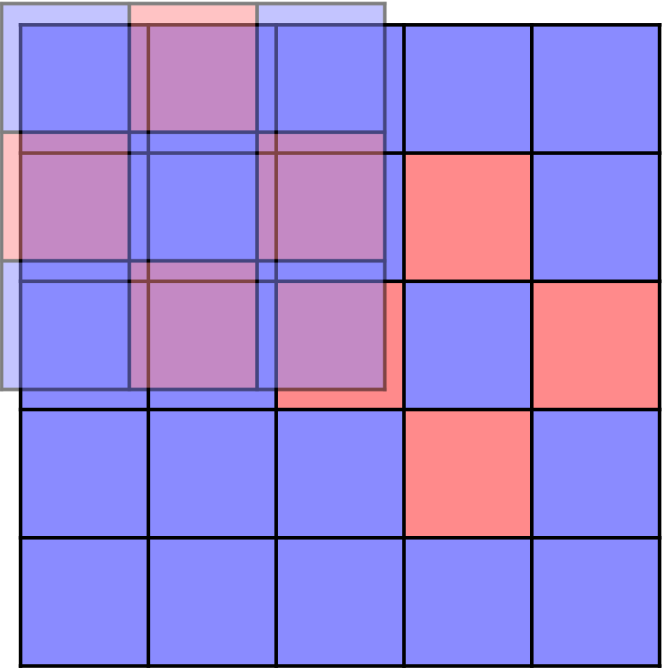
-1	-1	-1	-1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	1
-1	-1	-1	1	-1
-1	-1	-1	-1	-1

image

An Example...



filter

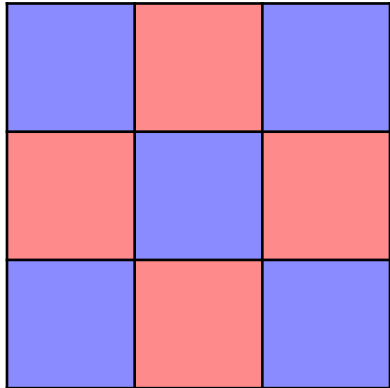


image

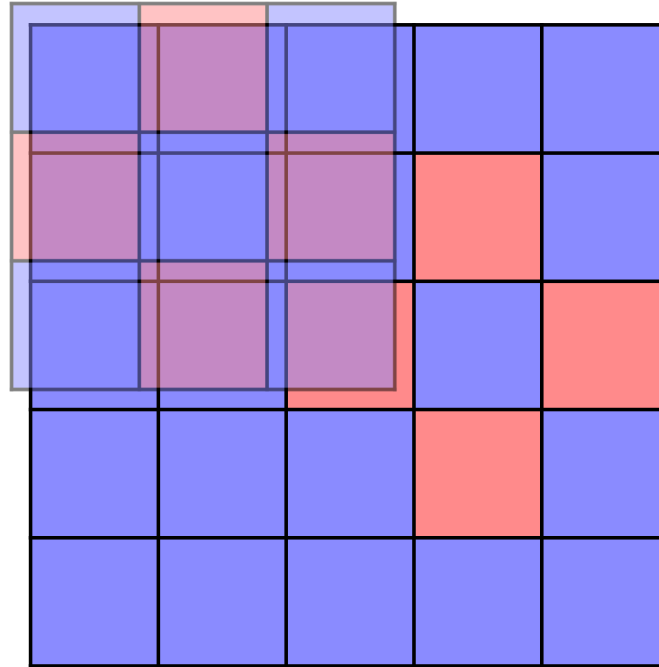
-1		

$$\mathbf{w}^T \mathbf{x}_i^R$$

An Example...



filter



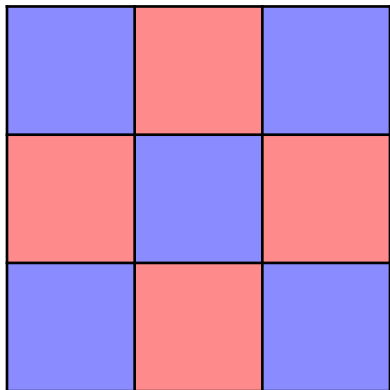
image

-1		

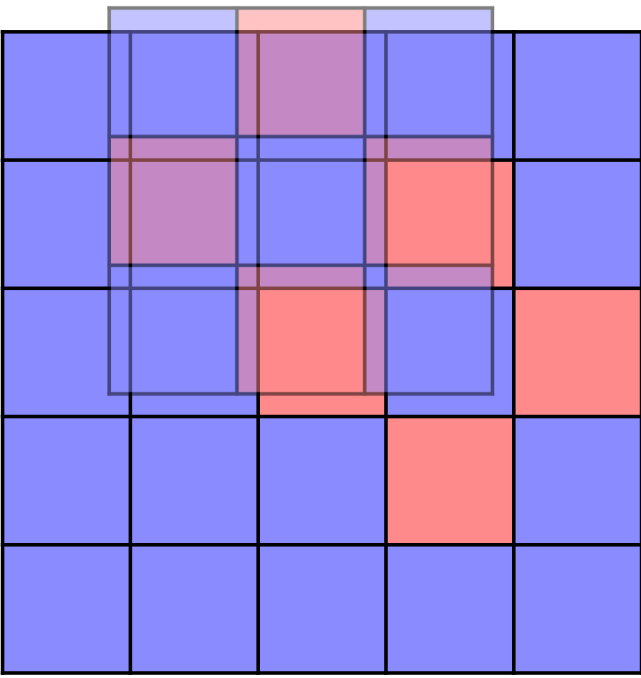
$$\mathbf{w}^T \mathbf{x}_i^R$$

Note: we're still using vector notation.
Alternatively, view as element-wise
product of matrices

An Example...



filter

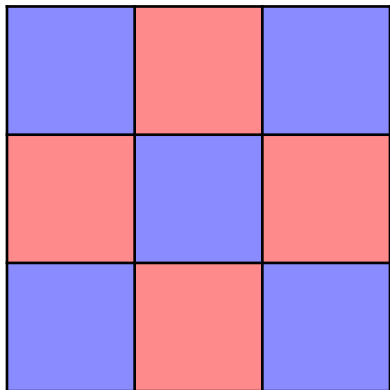


image

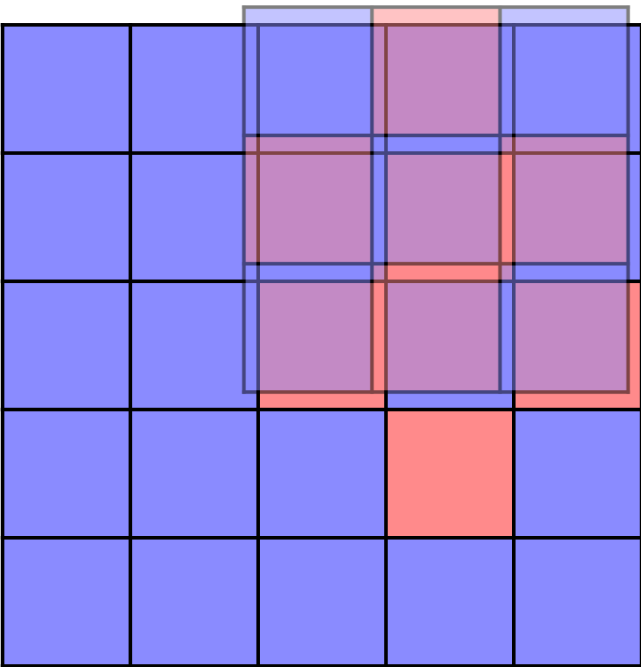
-1	5	

$$\mathbf{w}^T \mathbf{x}_i^R$$

An Example...



filter

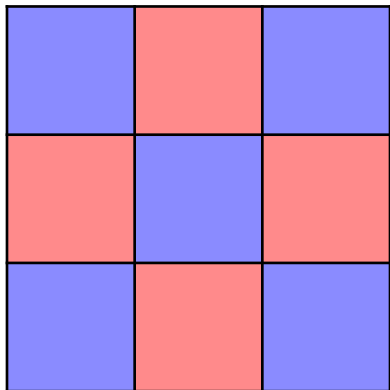


image

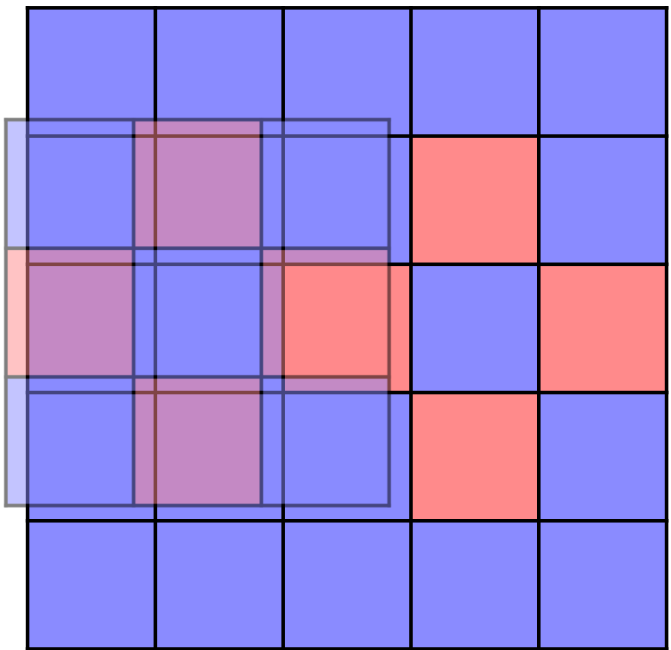
-1	5	-5

$$\mathbf{w}^T \mathbf{x}_i^R$$

An Example...



filter

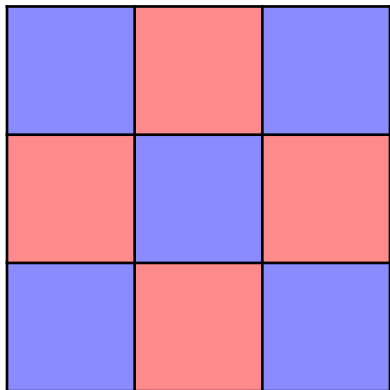


image

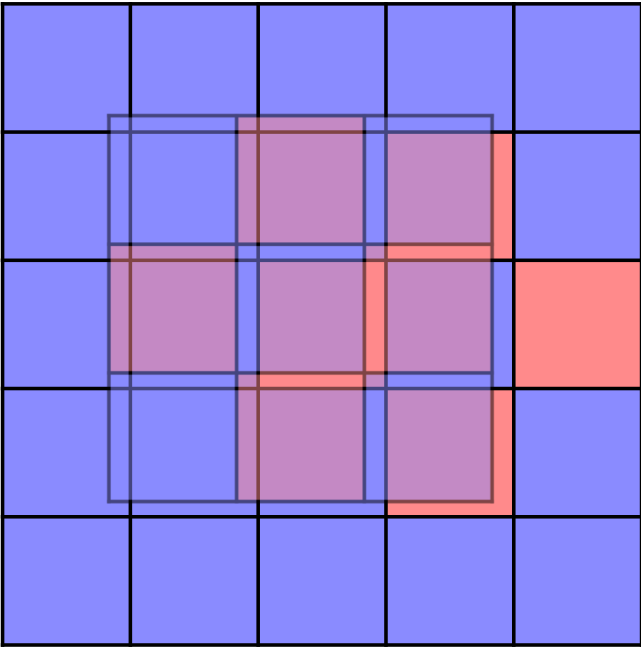
-1	5	-5
3		

$$\mathbf{w}^T \mathbf{x}_i^R$$

An Example...



filter

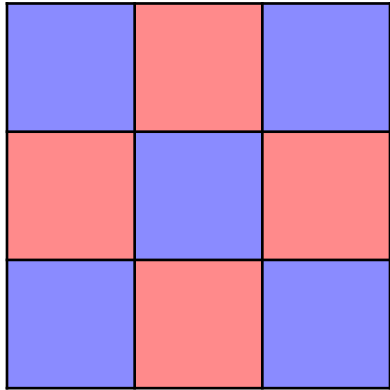


image

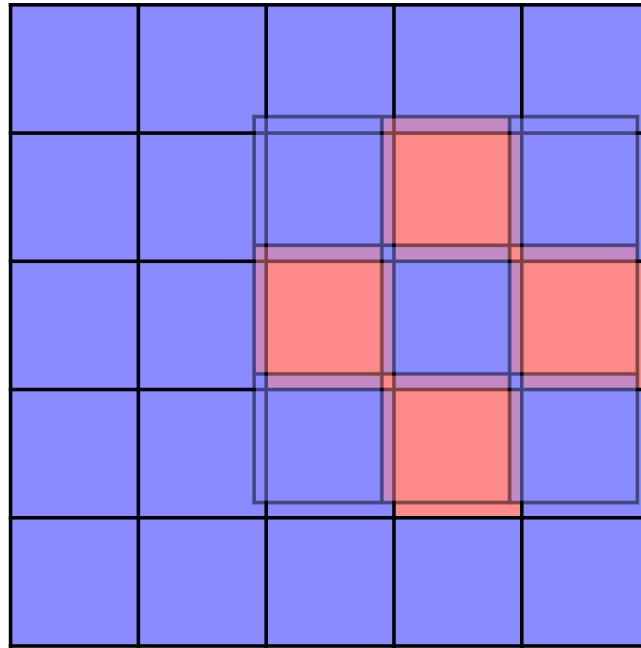
-1	5	-5
3	-5	

$$\mathbf{w}^T \mathbf{x}_i^R$$

An Example...



filter

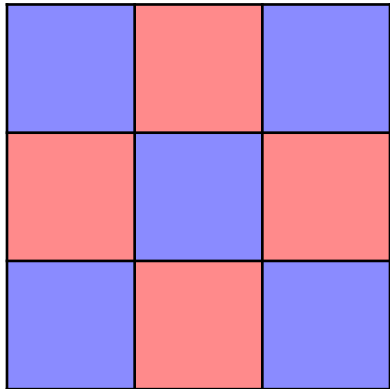


image

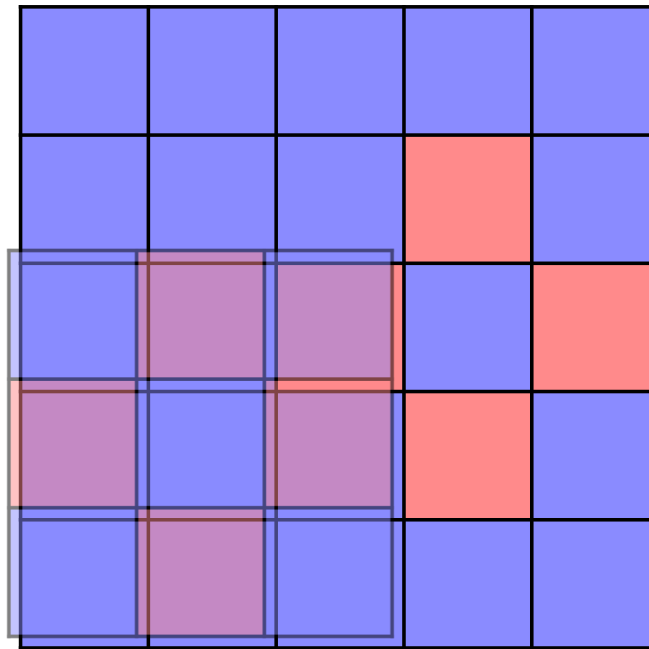
-1	5	-5
3	-5	9

$$\mathbf{w}^T \mathbf{x}_i^R$$

An Example...



filter

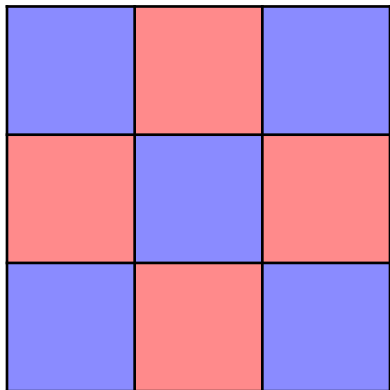


image

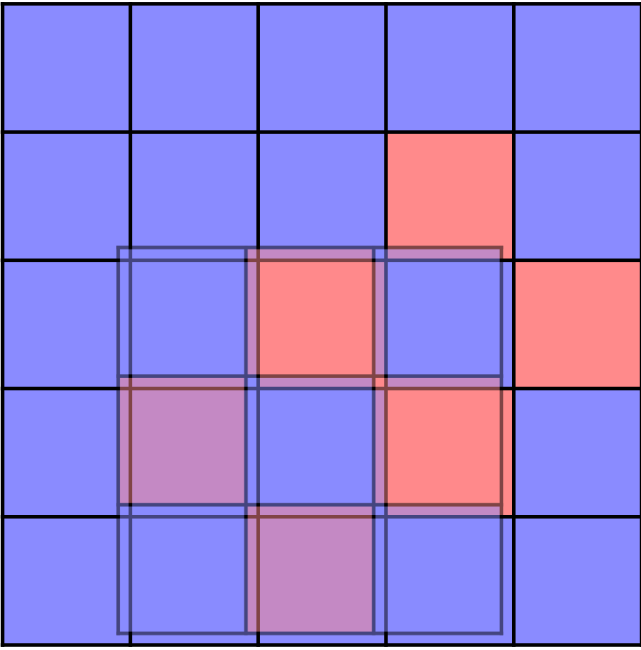
-1	5	-5
3	-5	9
-1		

$$\mathbf{w}^T \mathbf{x}_i^R$$

An Example...



filter

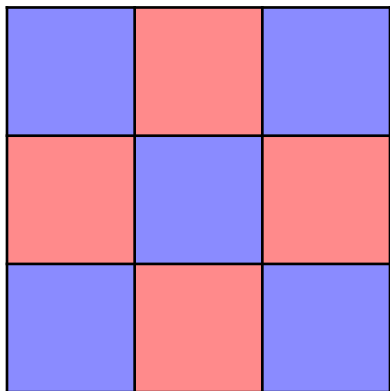


image

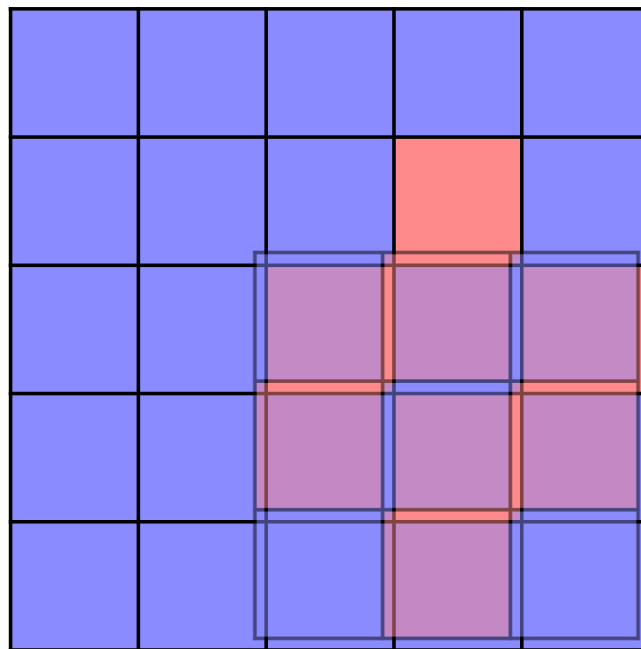
-1	5	-5
3	-5	9
-1	5	

$$\mathbf{w}^T \mathbf{x}_i^R$$

An Example...



filter

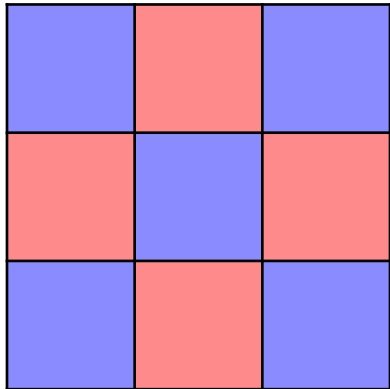


image

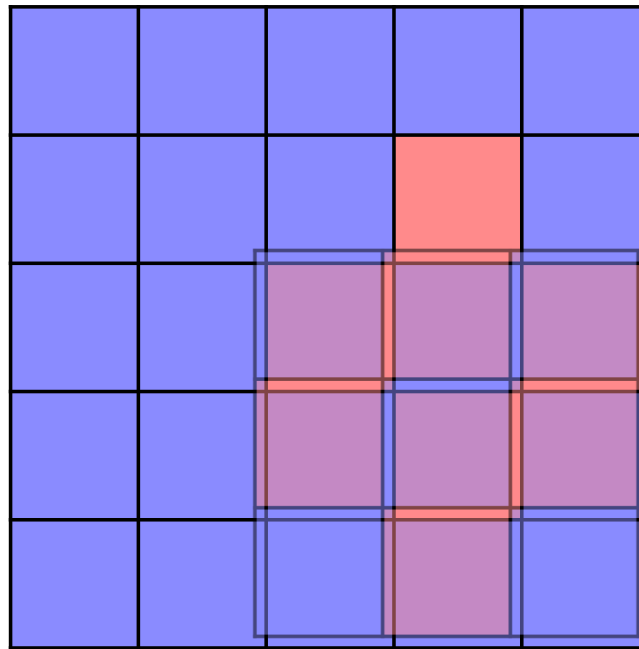
-1	5	-5
3	-5	9
-1	5	-5

$$\mathbf{w}^T \mathbf{x}_i^R$$

An Example...



filter

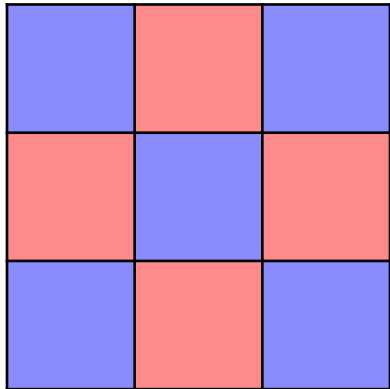


image

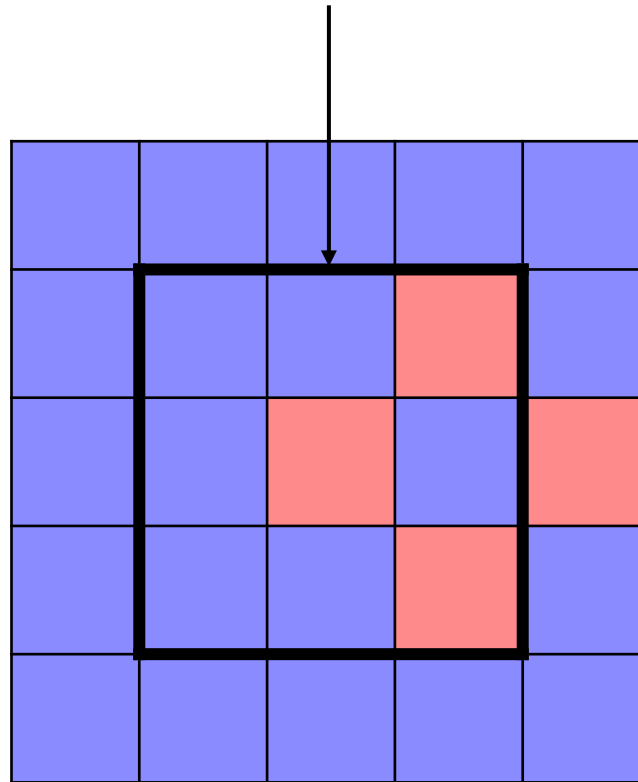
-1	5	-5
3	-5	9
-1	5	-5

$$\mathbf{w}^T \mathbf{x}_i^R$$

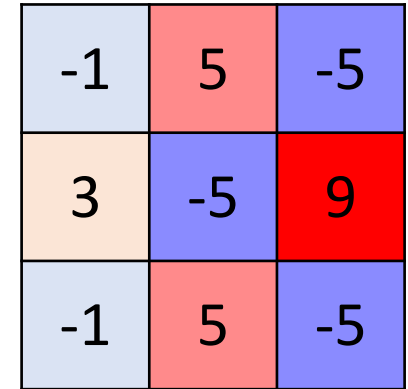
Each location where the filter was centered has been evaluated: “how similar is this location to the filter”?



filter



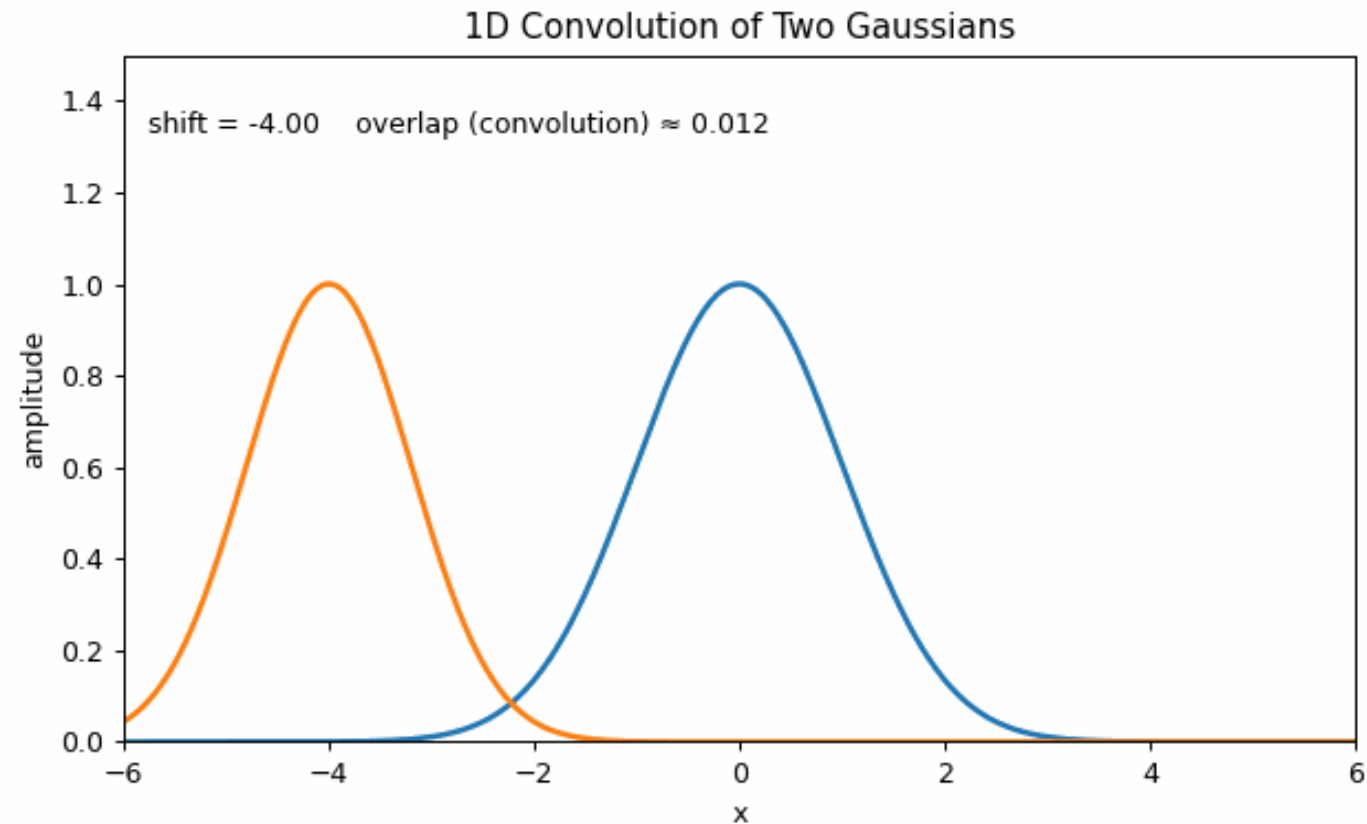
image



$$\mathbf{w}^T \mathbf{x}_i^R$$

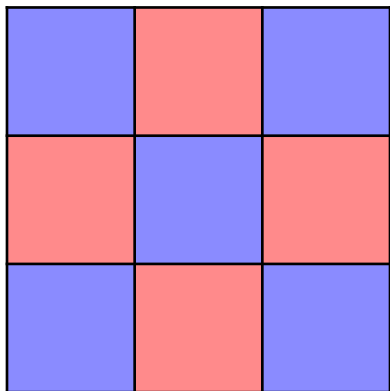
A note on the term “convolution”

- We commonly call the CNN operation “spatial convolution”
- However, “cross-correlation” is arguably more correct
- In particular, the kernel / filter is not flipped; however, this is irrelevant given that weights are learned

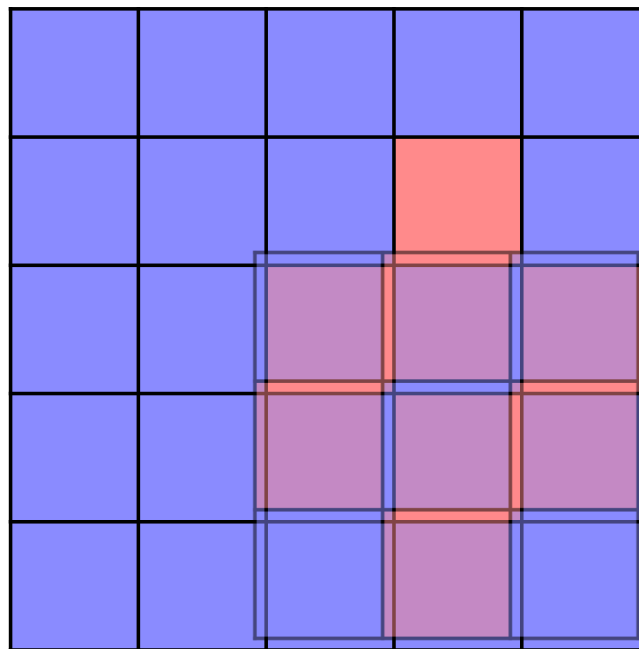


Padding and stride

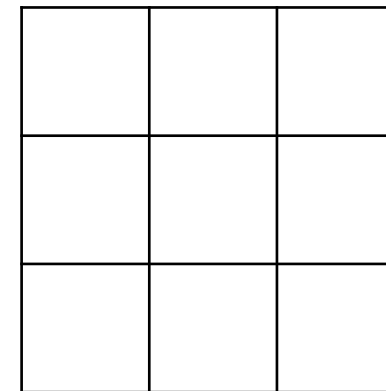
“Valid” convolutions (i.e., no padding)



filter

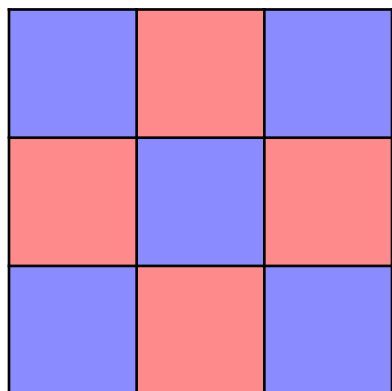


image

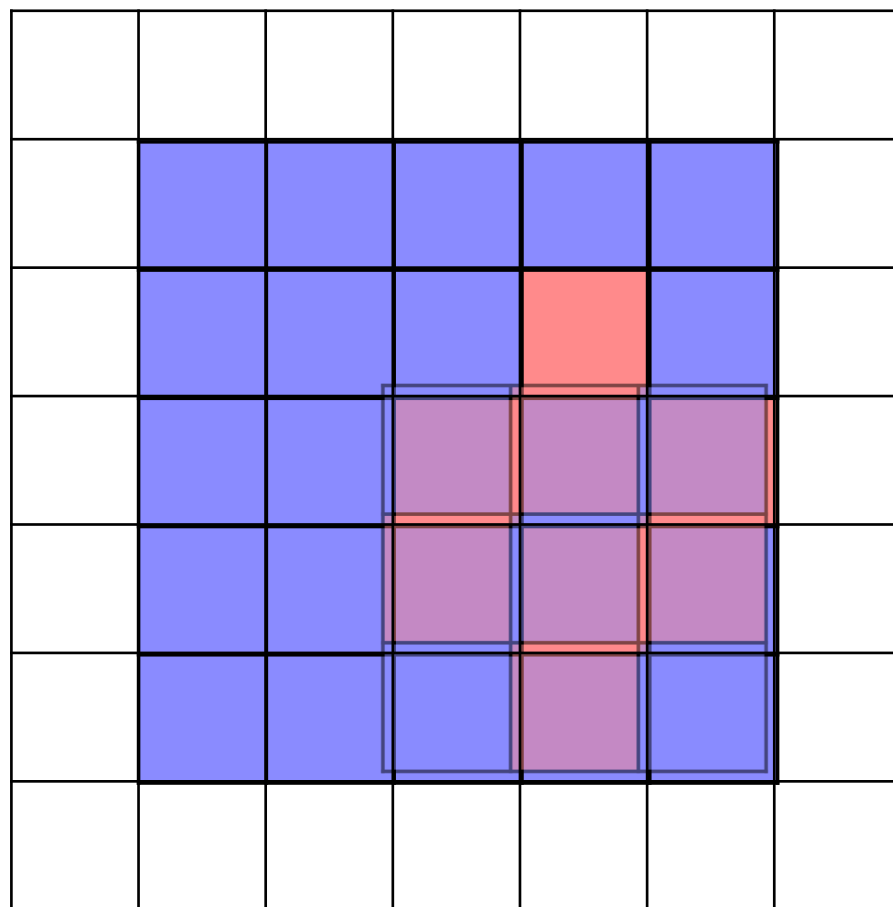


feature map

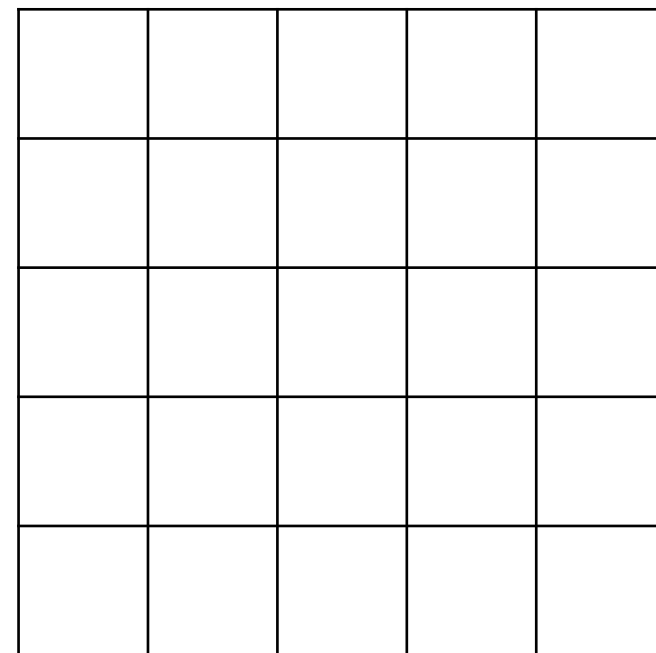
“Same” conv (i.e., pad to apply at each pixel)



filter

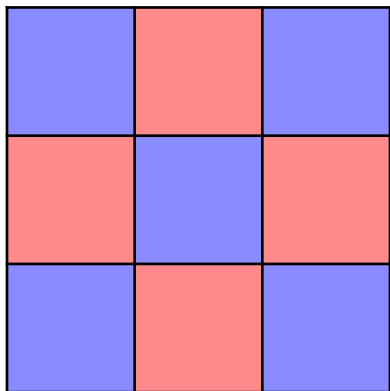


image

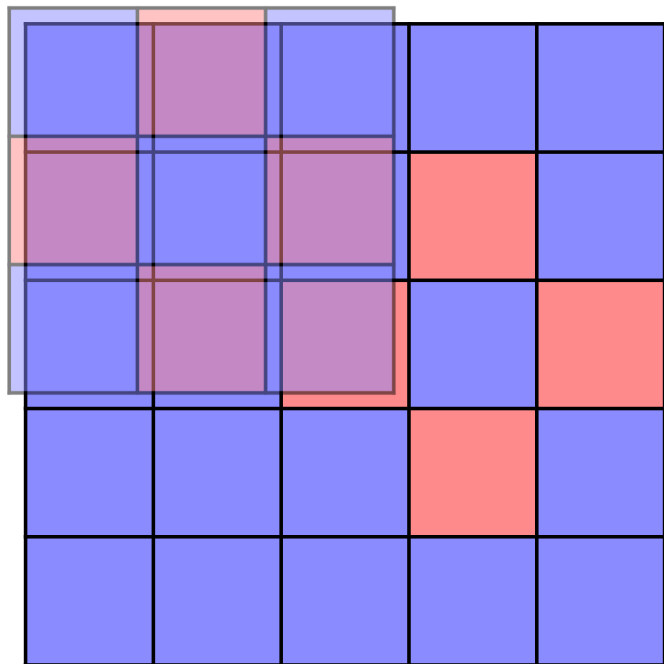


feature map

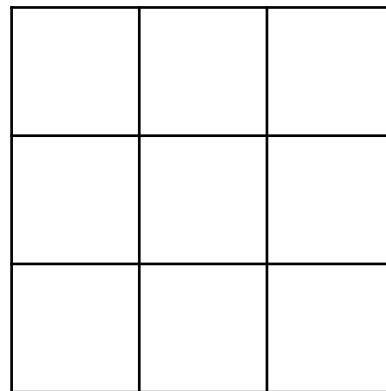
Stride 1



filter

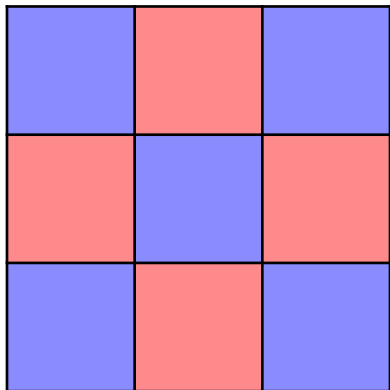


image

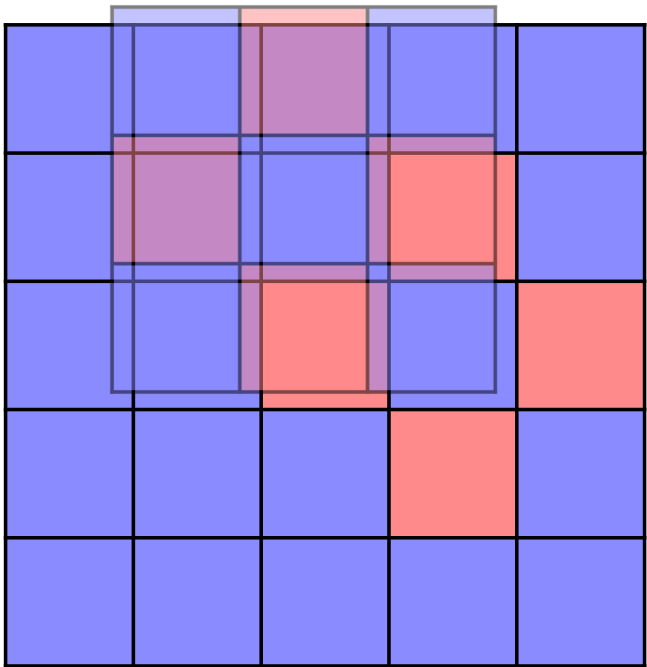


feature map

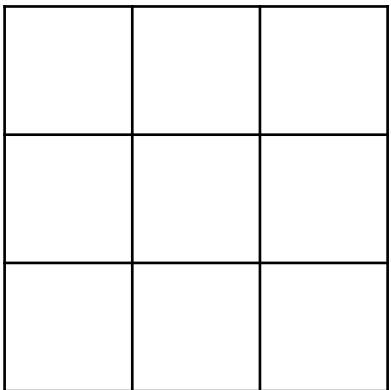
Stride 1



filter

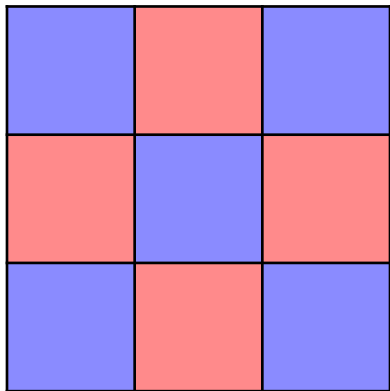


image

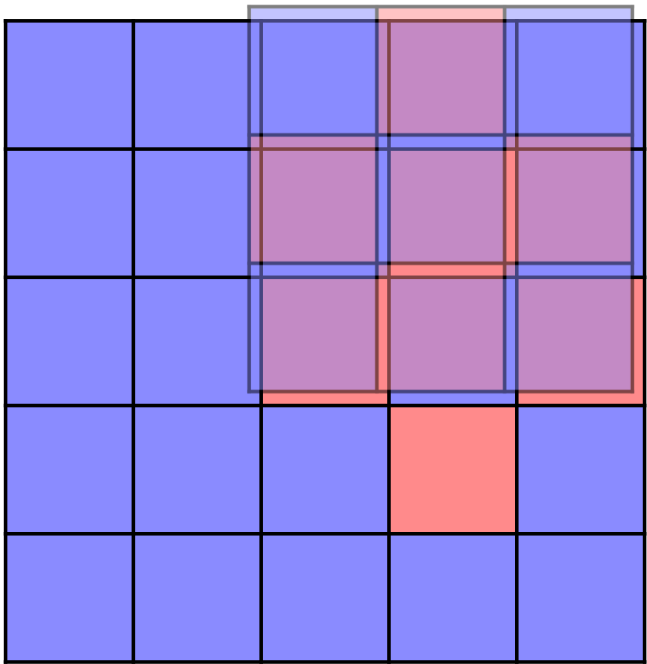


feature map

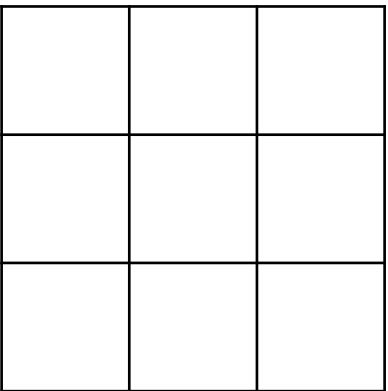
Stride 1



filter

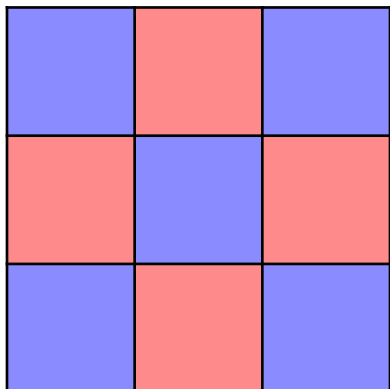


image

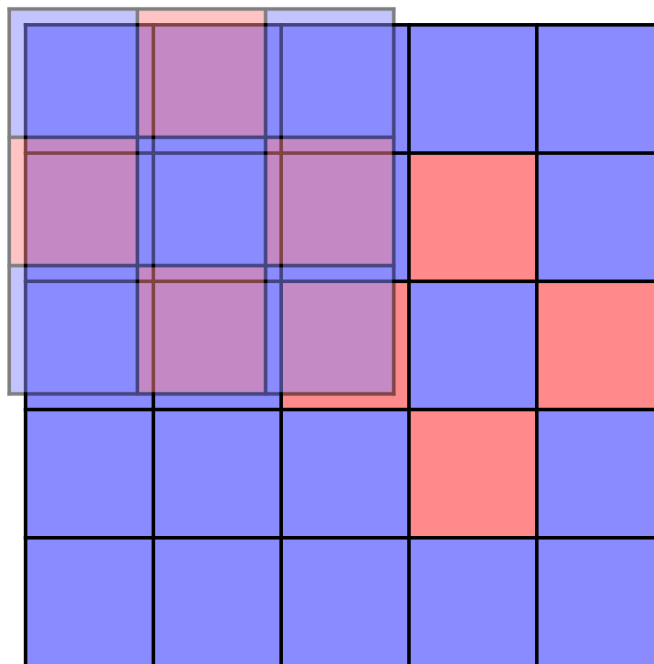


feature map

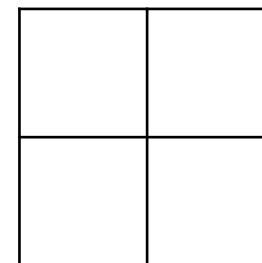
Stride 2



filter

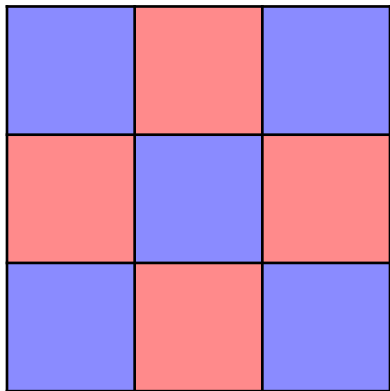


image

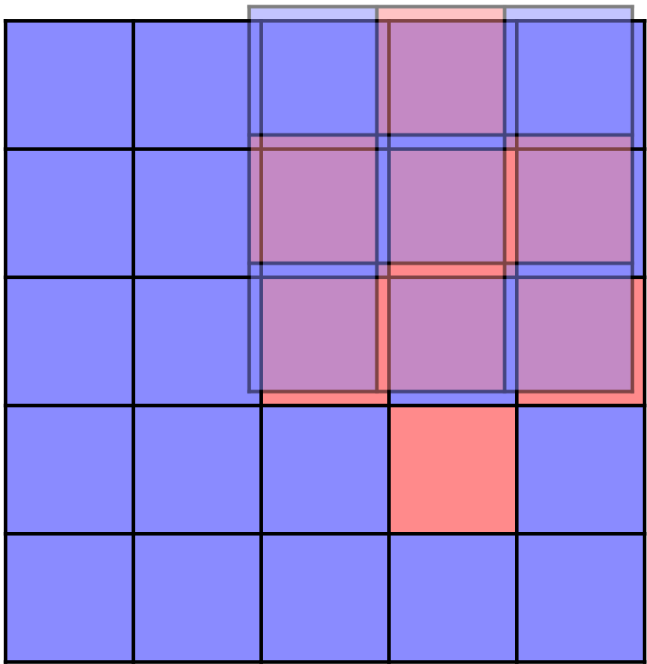


feature map

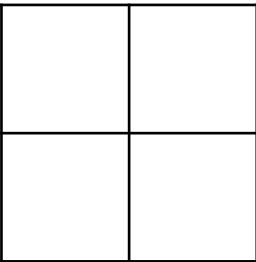
Stride 2



filter

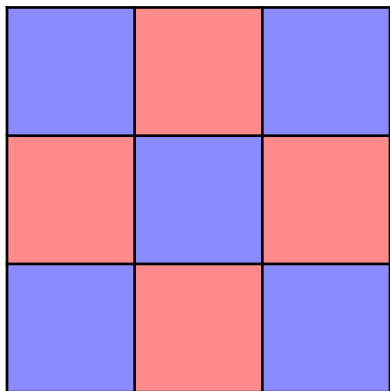


image

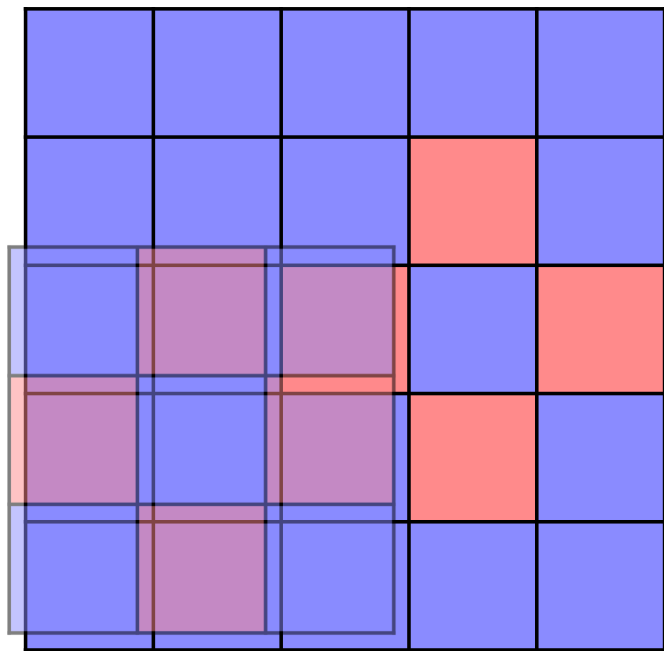


feature map

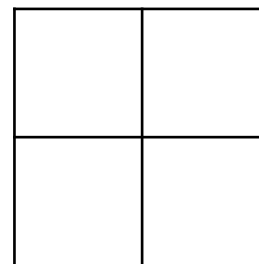
Stride 2



filter

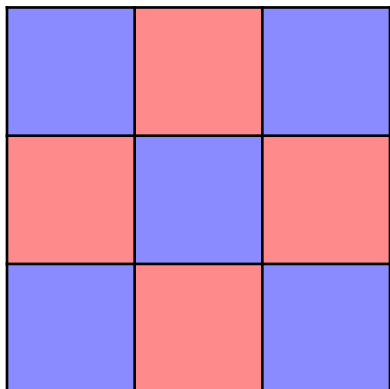


image

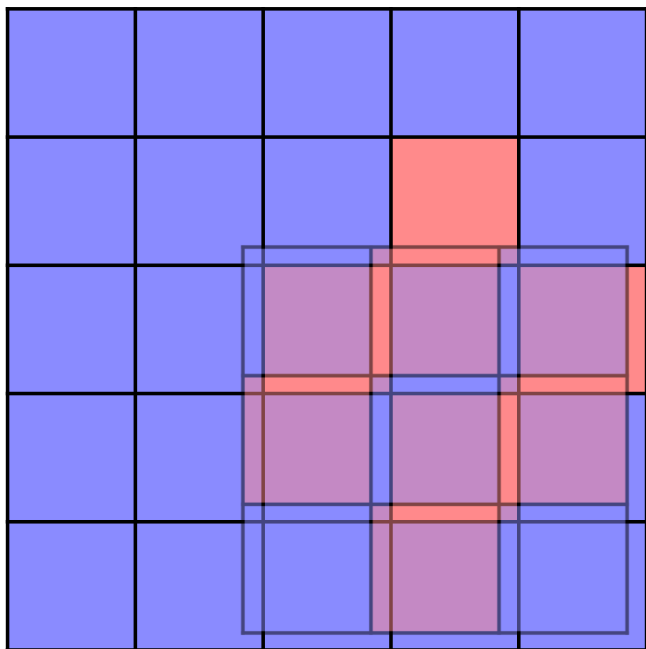


feature map

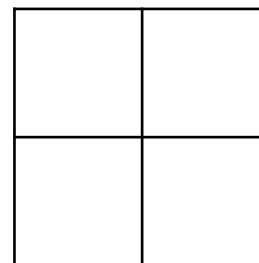
Stride 2



filter



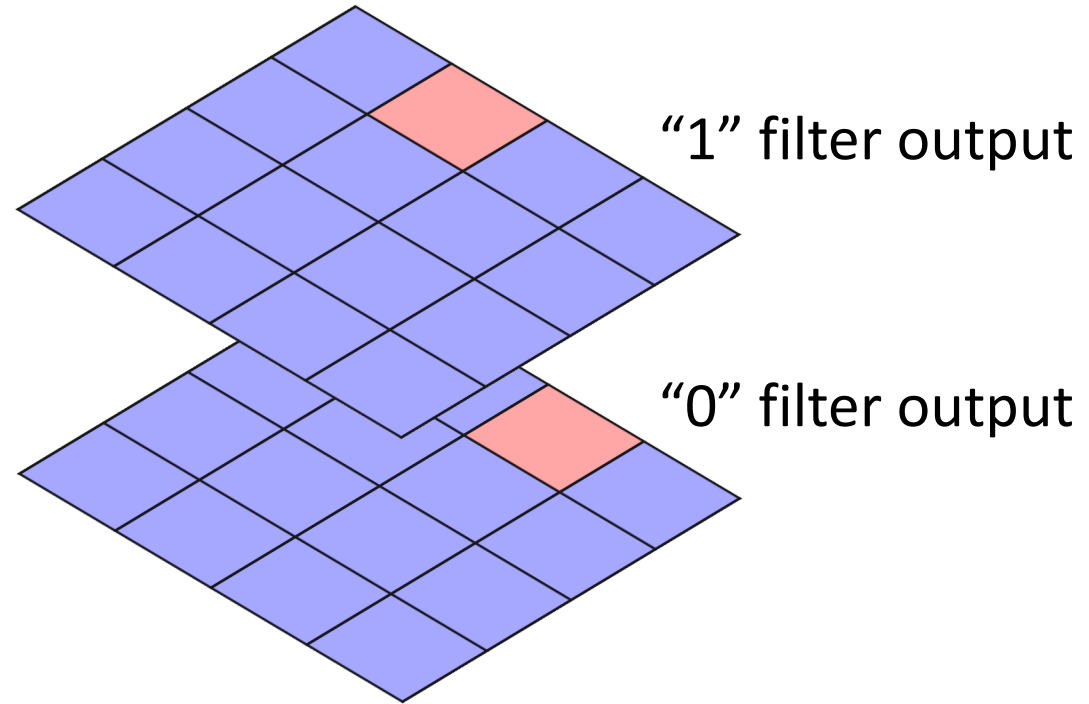
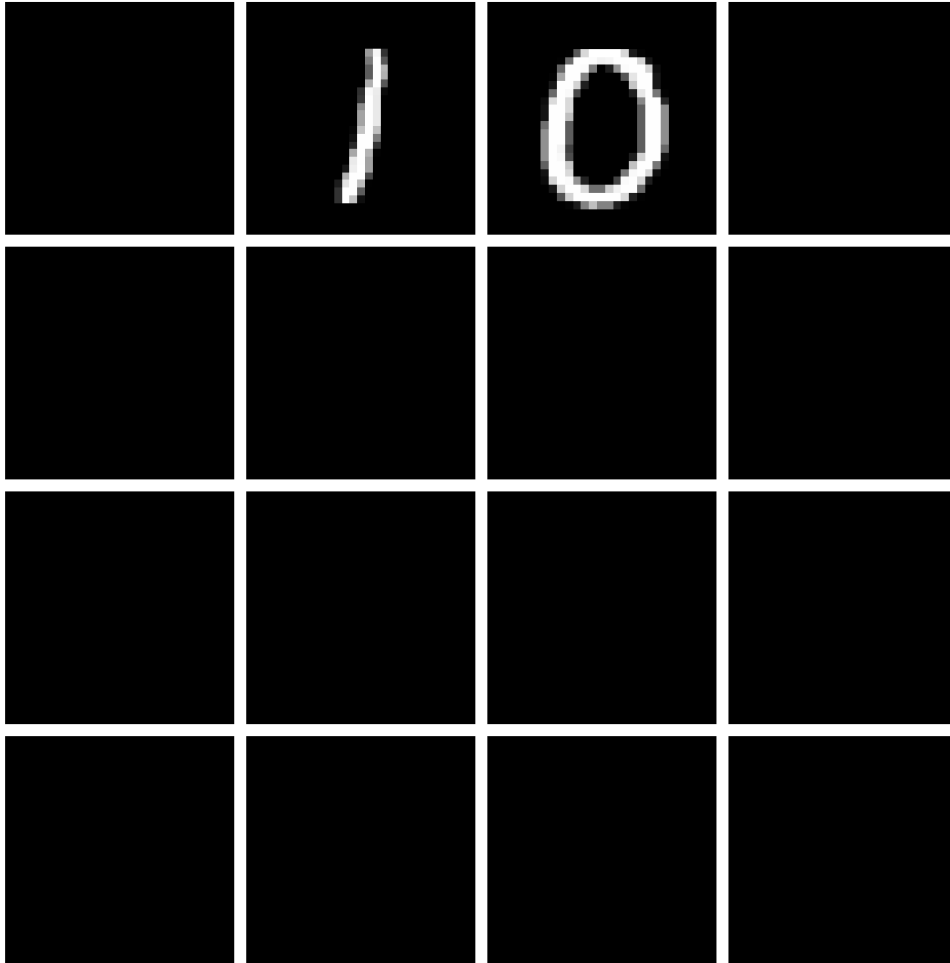
image



feature map

Convolutions on volumes

Convolutions on volumes

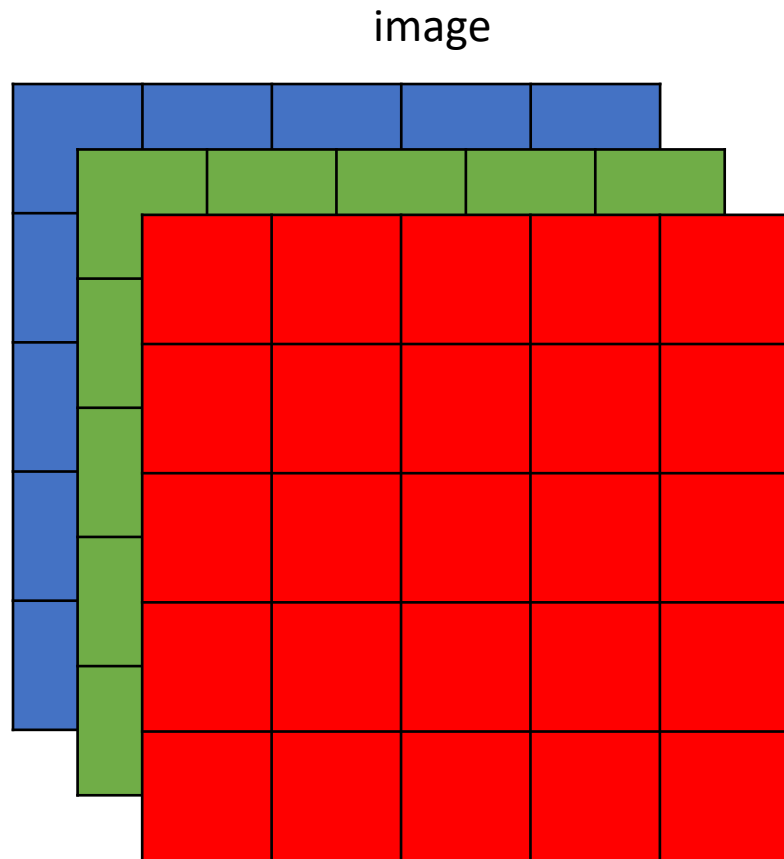


Layer 1: detect 1s and 0s

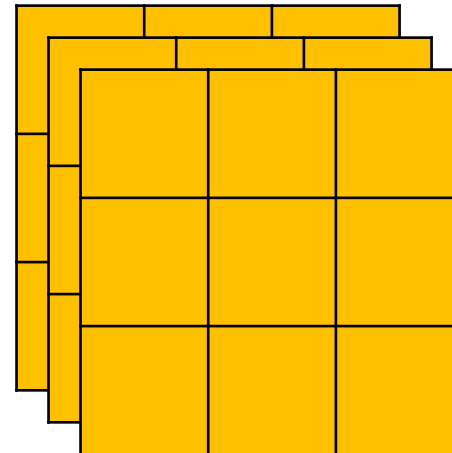
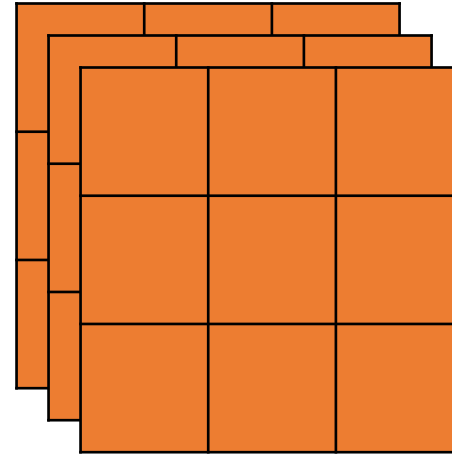
Layer 2: detect 10s (1 next to 0)

In this way, we learn to identify a hierarchy of features rather than a huge number of complex features

RGB images are volumes of pixels

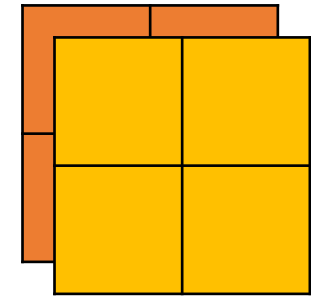


K filters of size $M \times N \times 3$



Output has depth K

(because there are
K feature maps)



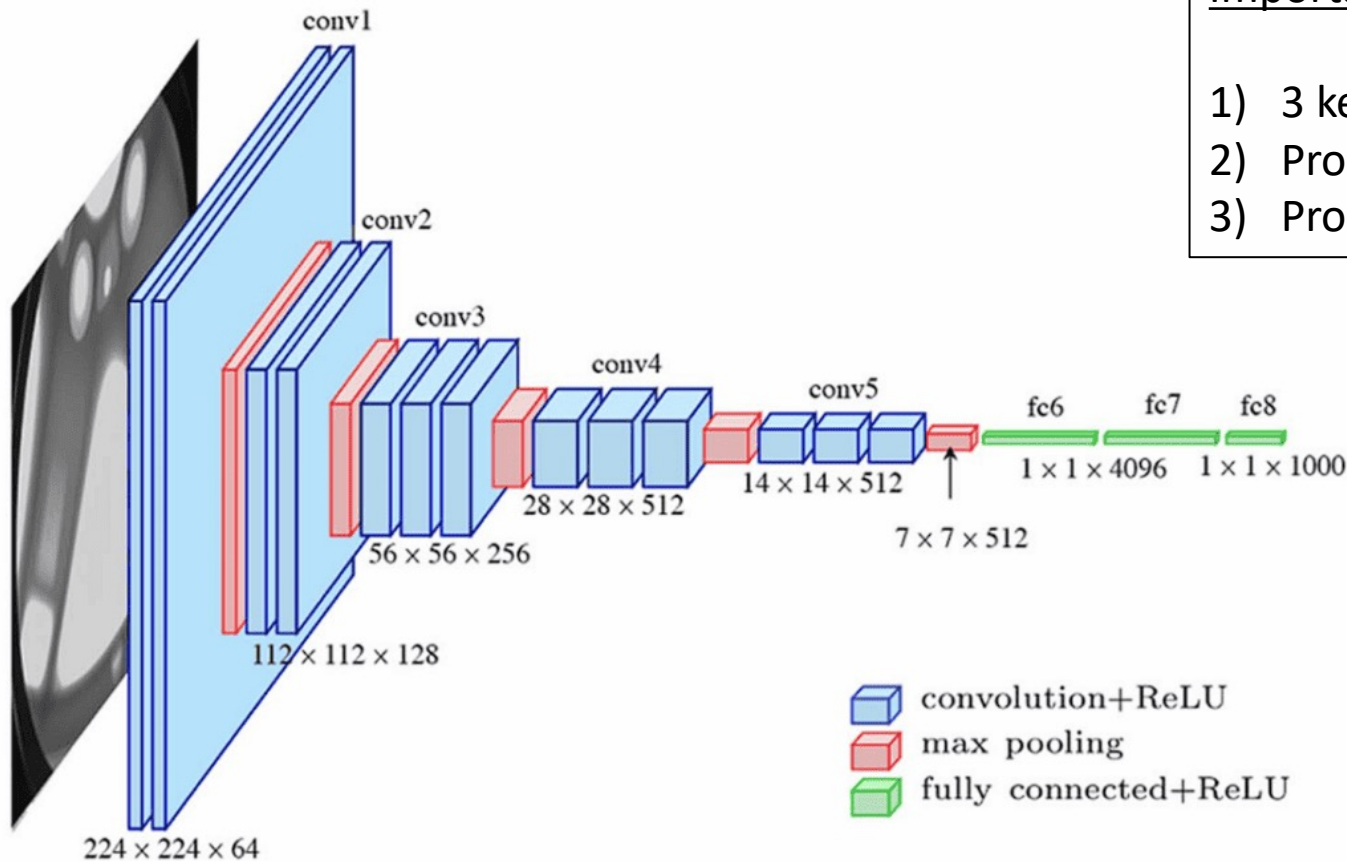
Here we have assumed:

- 1) No padding (i.e. "valid")
- 2) Stride 2 $\rightarrow$ (2 x 2) maps

Example architecture: VGG

Later we will discuss more (& more recent) architectures

Example architecture: VGG



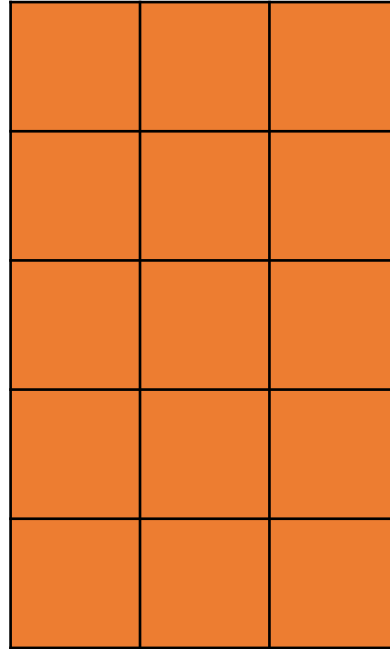
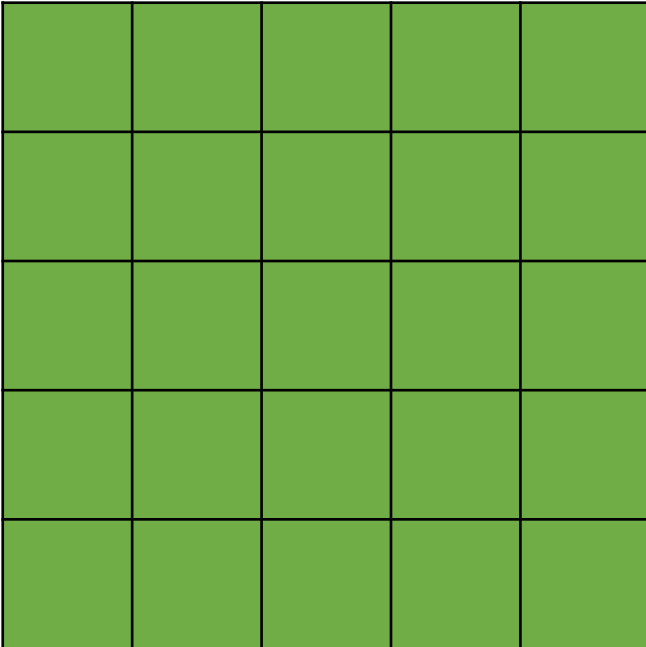
Important notes:

- 1) 3 key types of layers
- 2) Progressive decrease in spatial resolution
- 3) Progressive increase in number of features

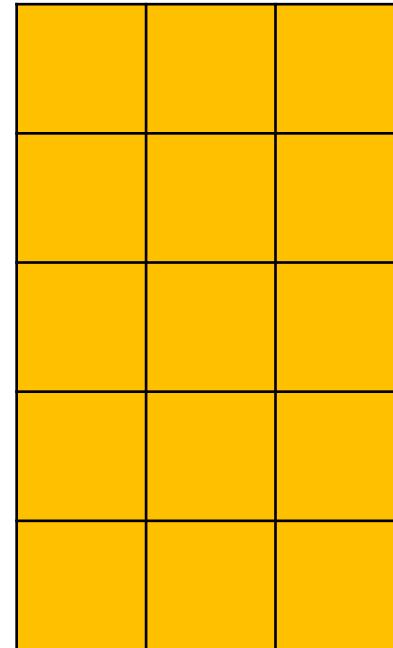
CNNs aren't just for images

CNNs for NLP and time series

$(L \times d)$ sequence



K filters of
size $(M \times d)$



Output has depth K

$[(L-M+1) \times K]$
for valid convolutions]

Report back (time permitting)

Calculate the size of the resulting feature maps as a function of:

- Input dimensions
- Padding scheme (valid, same)
- Stride